REDES NEURAIS

MULTILAYER PERCEPTRON

Prof. Tacla UTFPR/Curitiba

Multilayer Perceptron (MLP)

REDES MULTICAMADAS DE PERCEPTRONS

São várias camadas de neurônios sendo que as saídas dos neurônios de uma camada são utilizadas como entradas de outras camadas.

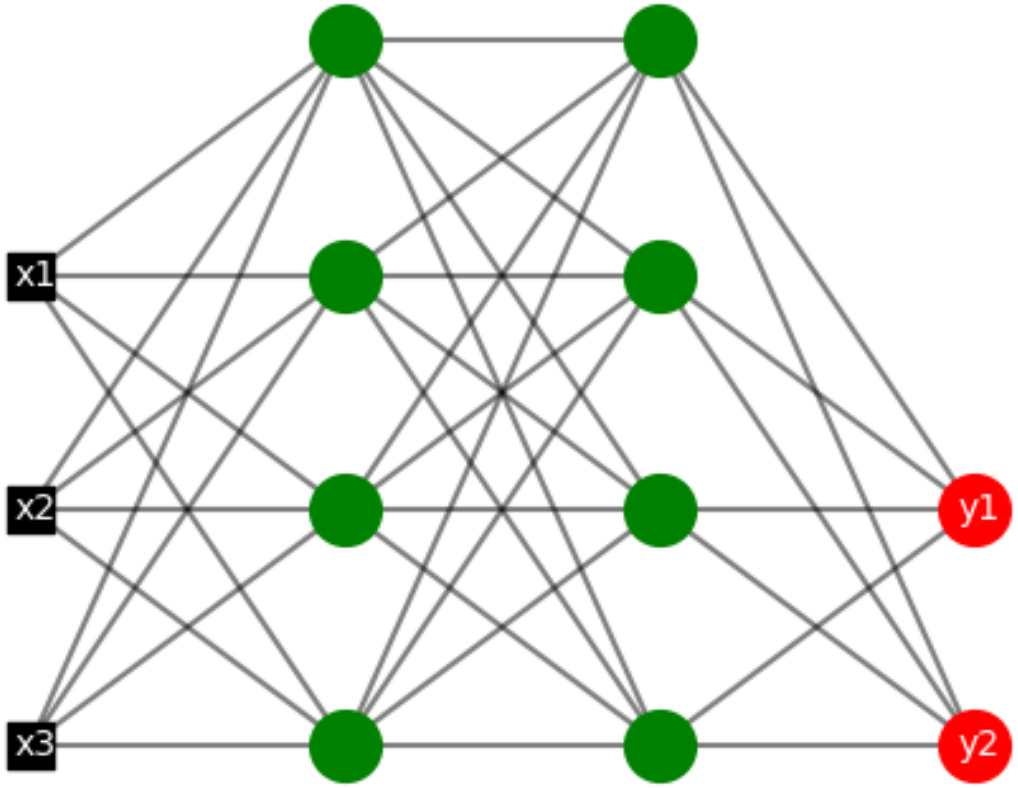
(Rumelhart et al., 1986) tornaram possível o uso de múltiplas camadas graças ao algoritmo de aprendizado denominado backpropagation, https://doi.org/10.1037%2Fh0042519

Características das MLPs

- Composta por **uma ou mais camadas escondidas** entre as camadas de entrada e saída
- Capaz de realizar **separações não-lineares**
- Pertence à arquitetura *feedforward* de múltiplas camadas
    - **Entradas**: reais ou simbólicas
    - **Saída**: reais, binárias ou **multiclasses** (podemos usar diversos neurônios)
    - **Função de ativação**: deriváveis* (sig, tanH, ReLu)
    - **Treinamento**: supervisionado
    - **Algoritmo de aprendizado**: **Backpropagation**

*Para usar backpropagation é necessário utilizar funções deriváveis

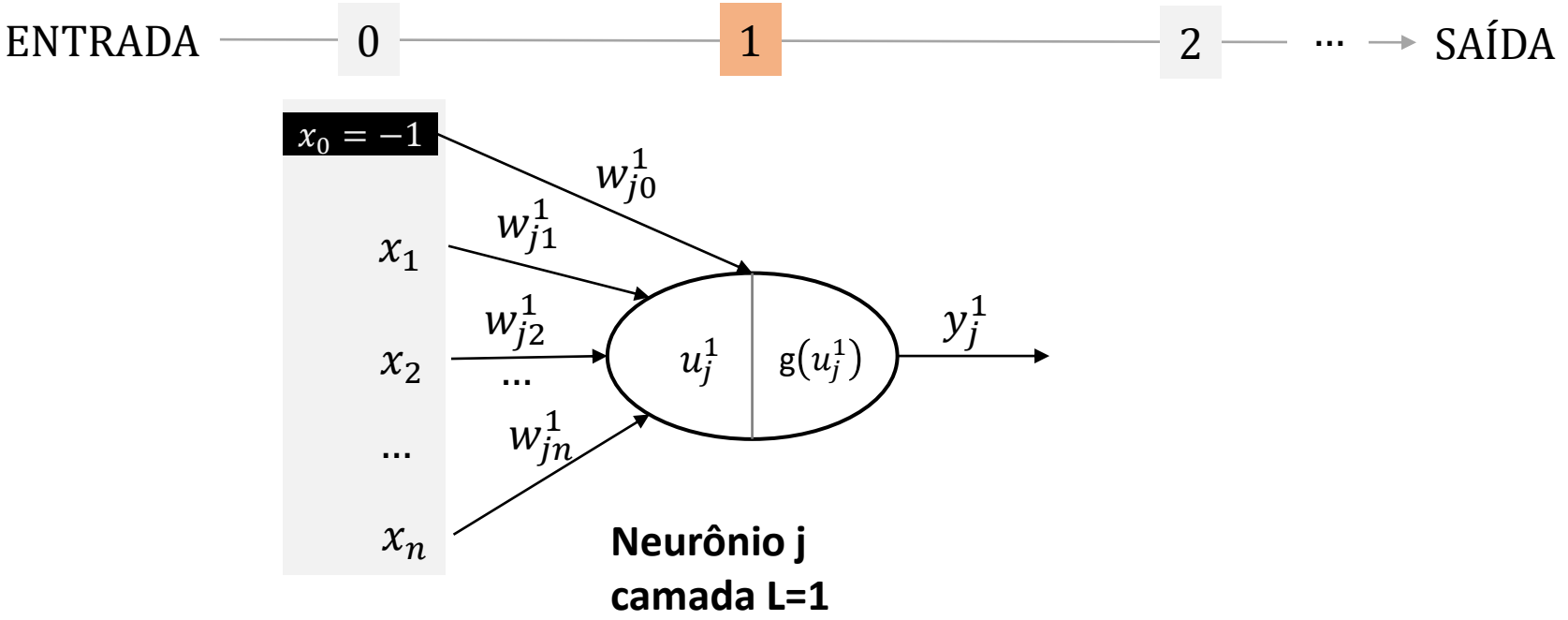
Representação gráfica de uma MLP



Camada de entrada Camadas escondidas Camada de saída

MLP: **Notação**

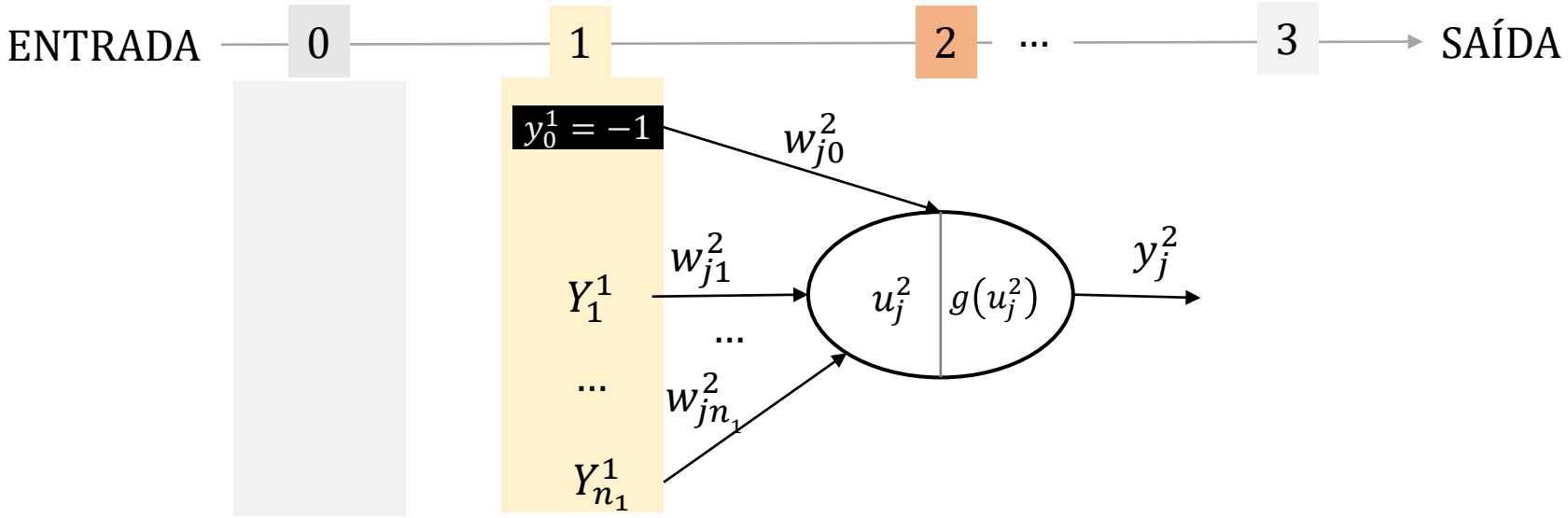
Camadas indexadas a partir de zero *(sobrescritos indicam a camada)*
n=total de neurônios na entrada, n_1=total de neurônios no layer 1, ...



u_j^L= potencial de ativação do neurônio j da camada L
$g(u_j^L)$ = ativação do neurônio j da camada L
$y_j^L = g(u_j)$ saída do neurônio j da camada L
w_{ji}^L= peso que conecta a saída do neurônio i da camada (L – 1) ao neurônio j da camada L

MLP: Notação – potencial de ativação

Um neurônio qualquer da camada 2



$$u_j^L = \sum_{i \in n_{L-1}} w_{ji}^L \, y_i^{L-1}$$

Potencial de ativação para um neurônio j qualquer da camada L

n_{L-1} = número de neurônios da camada anterior

MLP: **Notação – rede neural**

matriz de pesos

$$W_{ji}^2 = \begin{bmatrix} w_{10}^2 & w_{11}^2 & \dots & w_{1n_1}^2 \\ w_{20}^2 & w_{21}^2 & \dots & w_{2n_1}^2 \\ \dots & & & \\ w_{n_2 0}^2 & w_{n_2 1}^2 & \dots & w_{n_2 n_1}^2 \end{bmatrix}$$

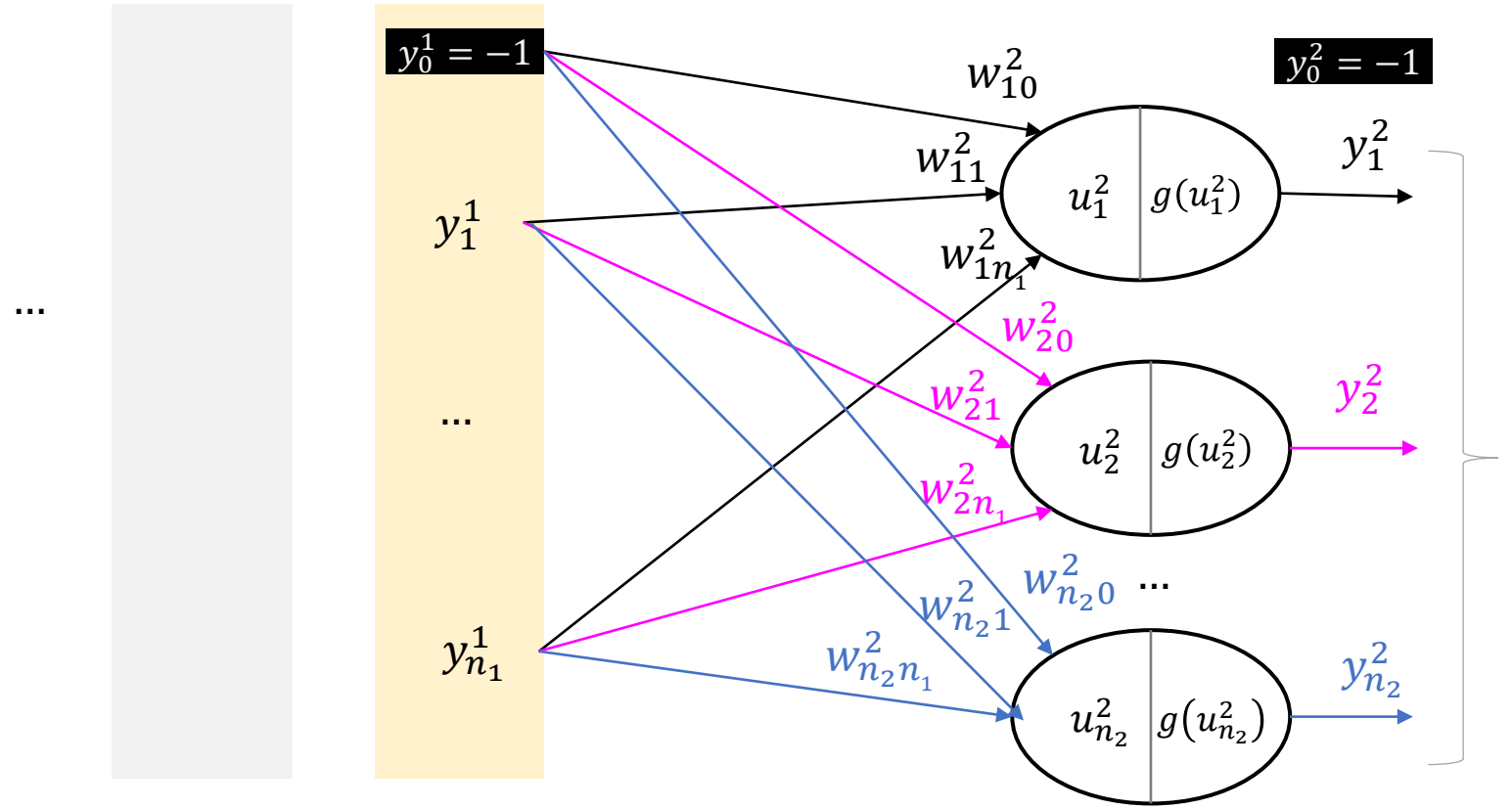
vetor pot. ativação

$$U_j^2 = \begin{bmatrix} u_1^2 \\ u_2^2 \\ \dots \\ u_{n_2}^2 \end{bmatrix}$$

vetor das saídas da camada 2

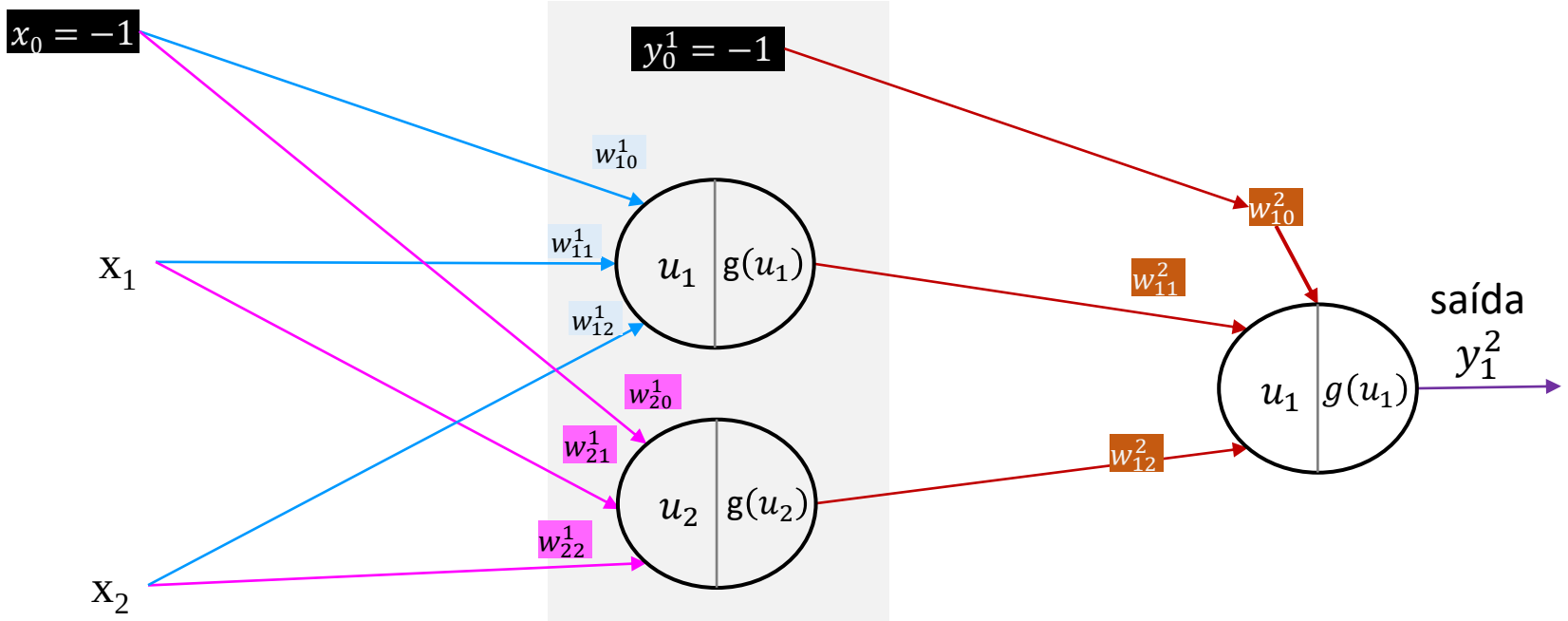
$$Y_j^2 = \begin{bmatrix} y_0^2 \\ y_1^2 \\ y_2^2 \\ \dots \\ y_{n_2}^2 \end{bmatrix}$$

ENTRADA 0 — 1 — 2 — … — 3 → SAÍDA



MLP: **Exemplo XOR**

RN XOR somente com perceptrons com $g(u)$ 'degrau' para todos os neurônios



$$W_{ji}^1 = \begin{bmatrix} w_{10}^1 & w_{11}^1 & w_{12}^1 \\ w_{20}^1 & w_{21}^1 & w_{22}^1 \end{bmatrix}$$

$$W_{ji}^2 = [w_{10}^2 \quad w_{11}^2 \quad w_{12}^2]$$

matrizes de pesos por layer

CAMADA 0
ENTRADA

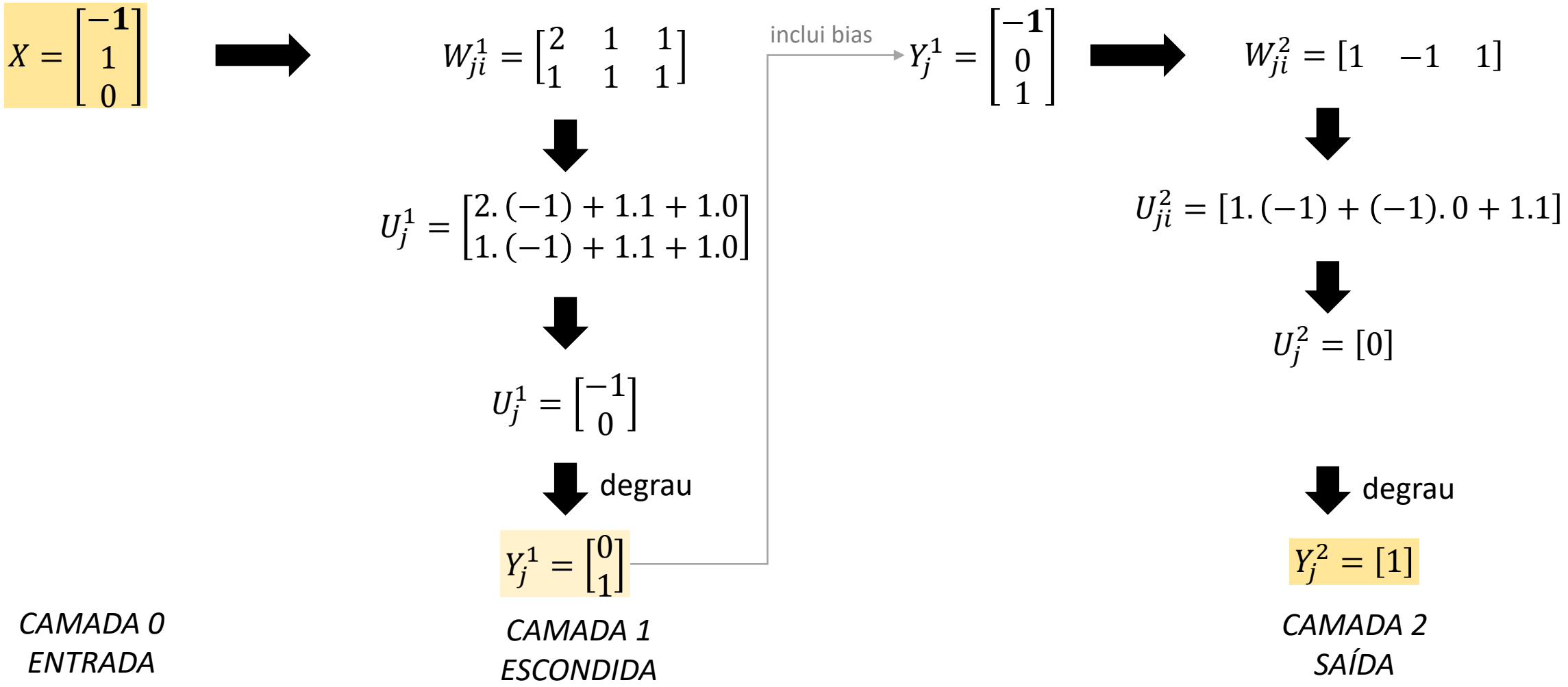
CAMADA 1
ESCONDIDA

CAMADA 2
SAÍDA

Transformação do espaço de entradas

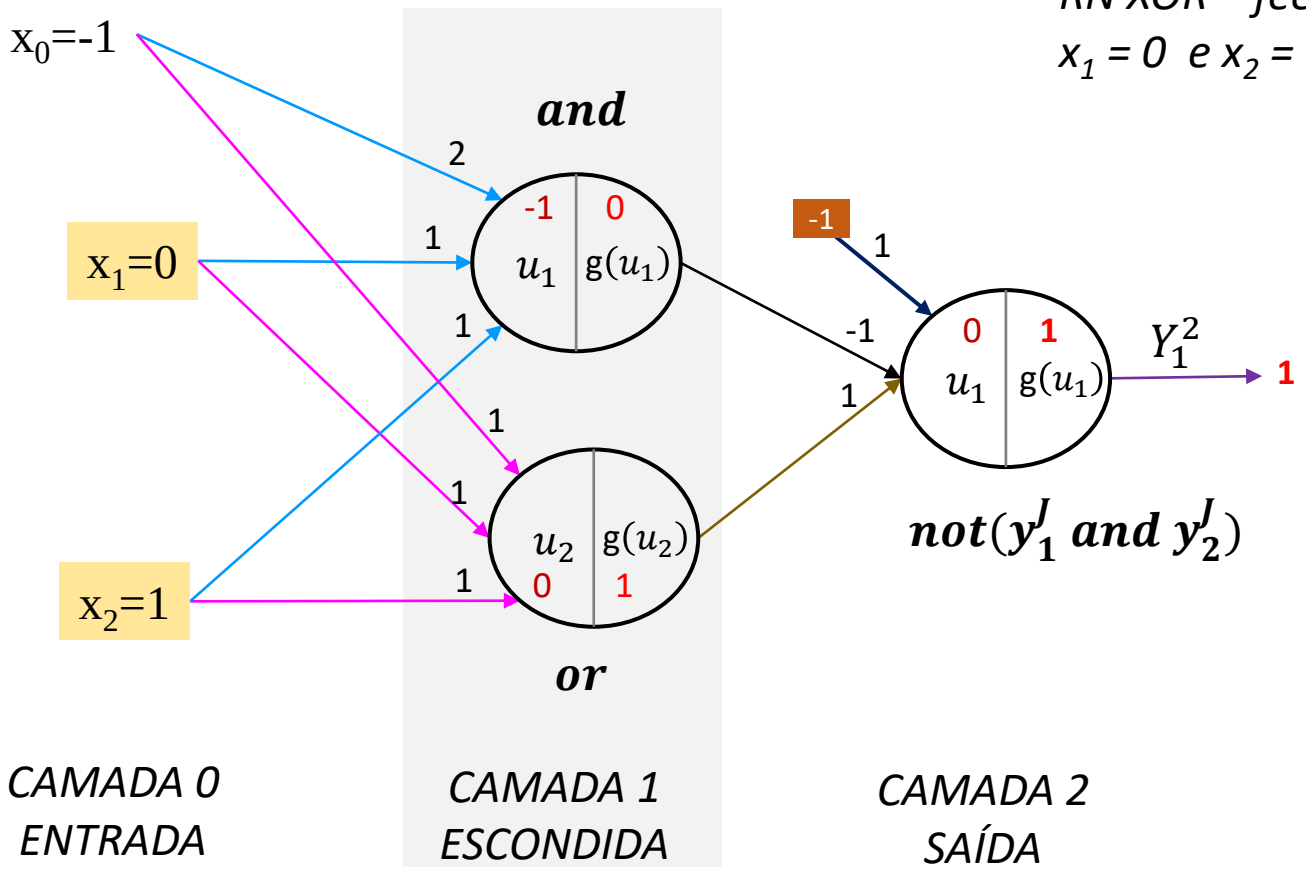
Uma MLP para resolver XOR

MLP: **Exemplo XOR**



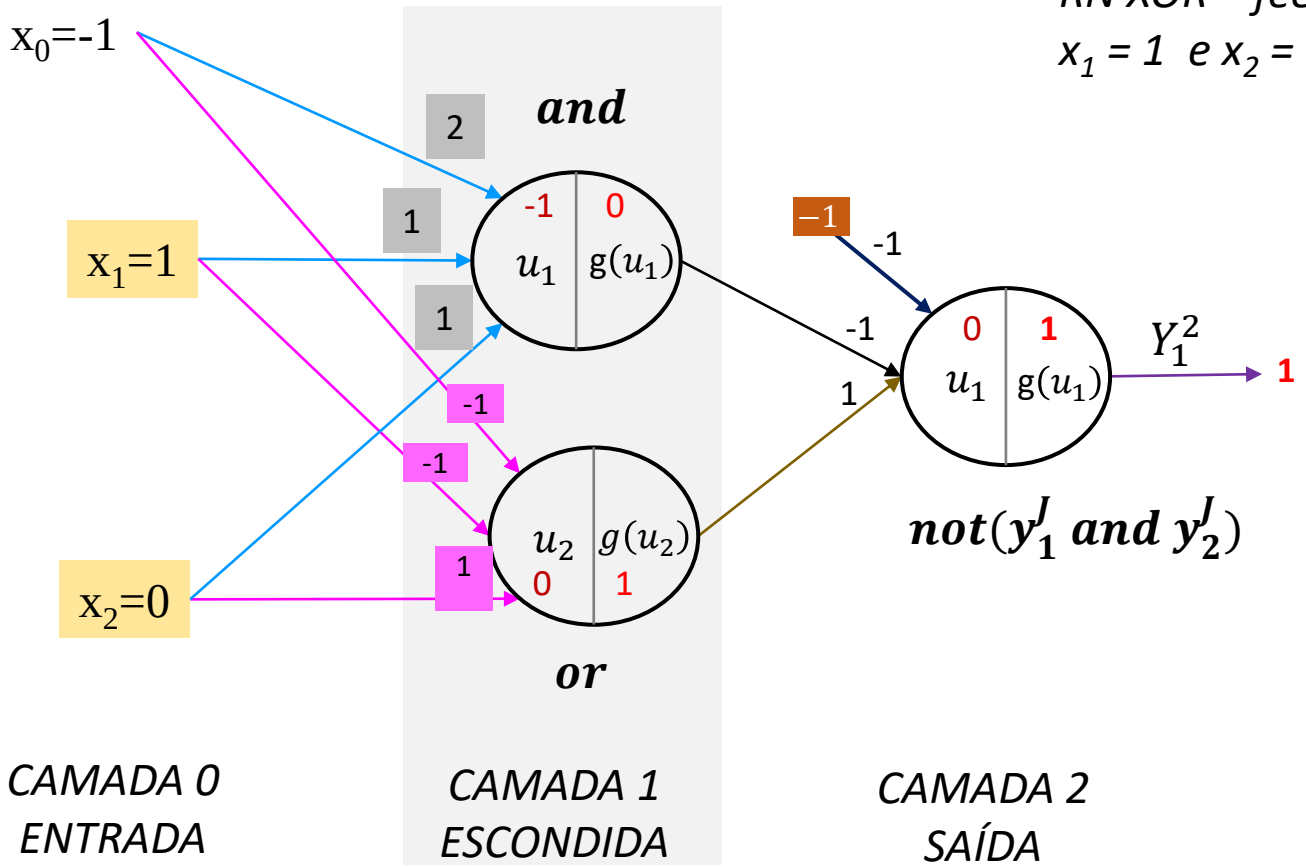
MLP: **Transformação espaço de entradas (XOR)**

RN XOR – feedforward para entradas
$x_1 = 0$ e $x_2 = 1$



MLP: **Transformação espaço de entradas (XOR)**

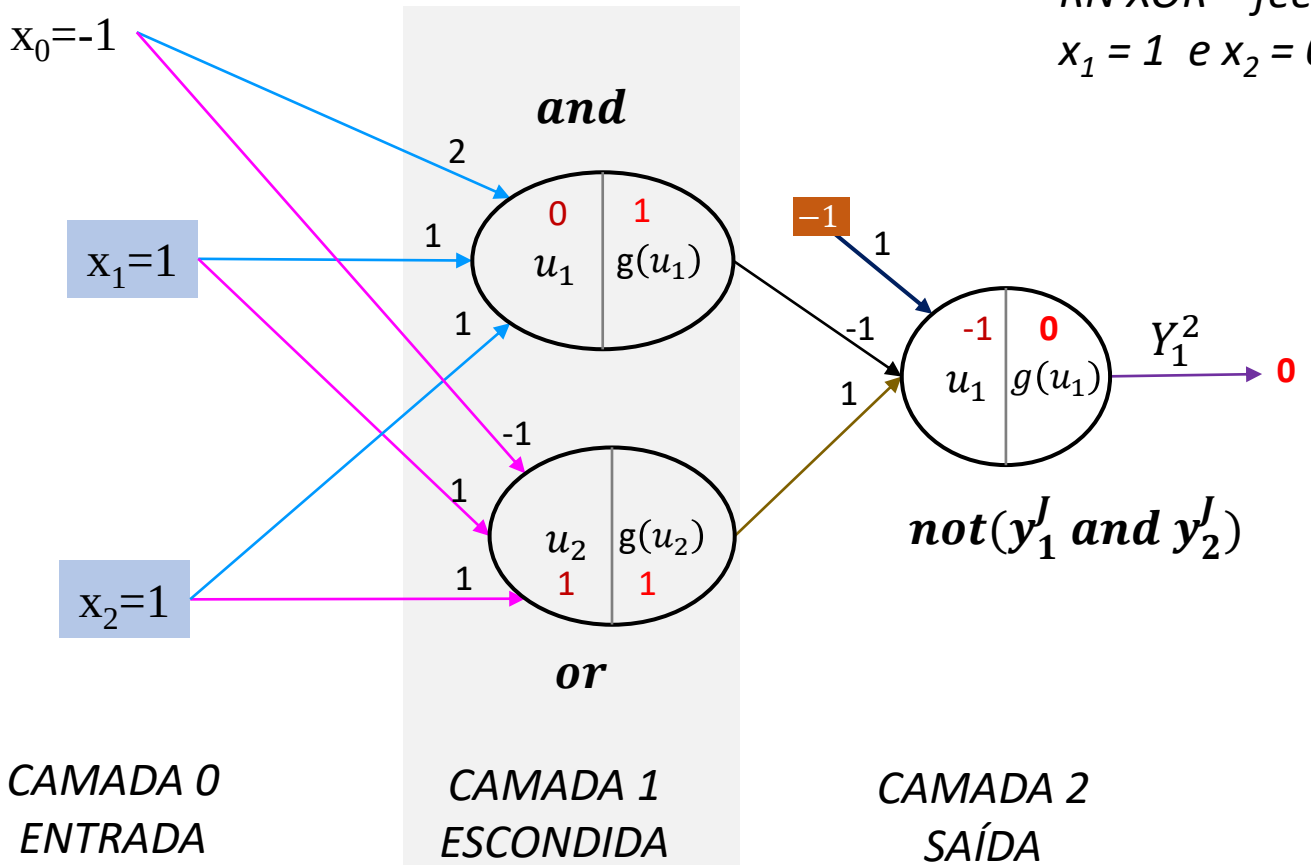
RN XOR – feedforward para entradas
$x_1 = 1$ e $x_2 = 0$



As saídas dos neurônios da camada 1 são iguais para a entrada x_1=0 e x_2=1

MLP: **Transformação espaço de entradas (XOR)**

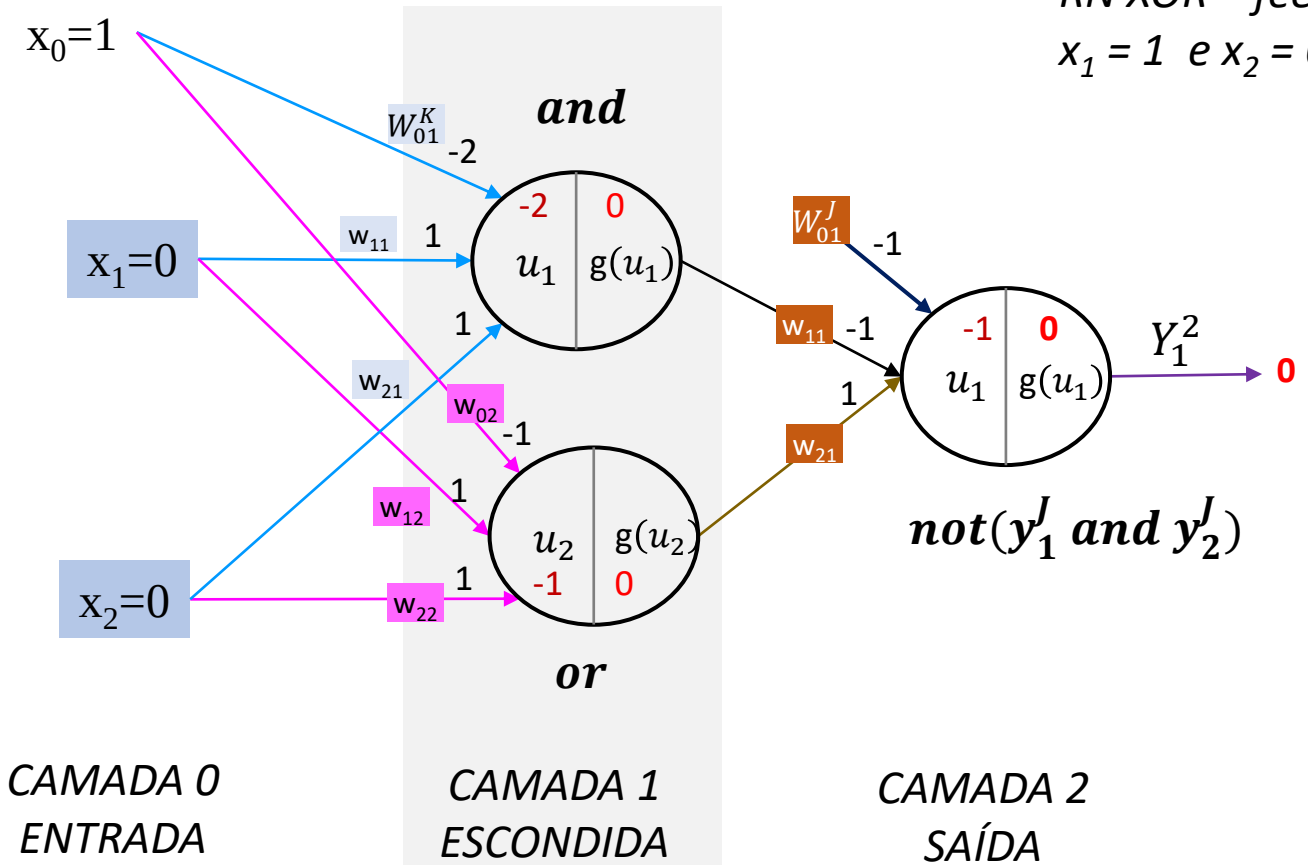
RN XOR – feedforward para entradas
x_1 = 1 e x_2 = 0



As saídas dos neurônios da camada 1 são iguais quando a entrada é x_1=0 e x_2=1

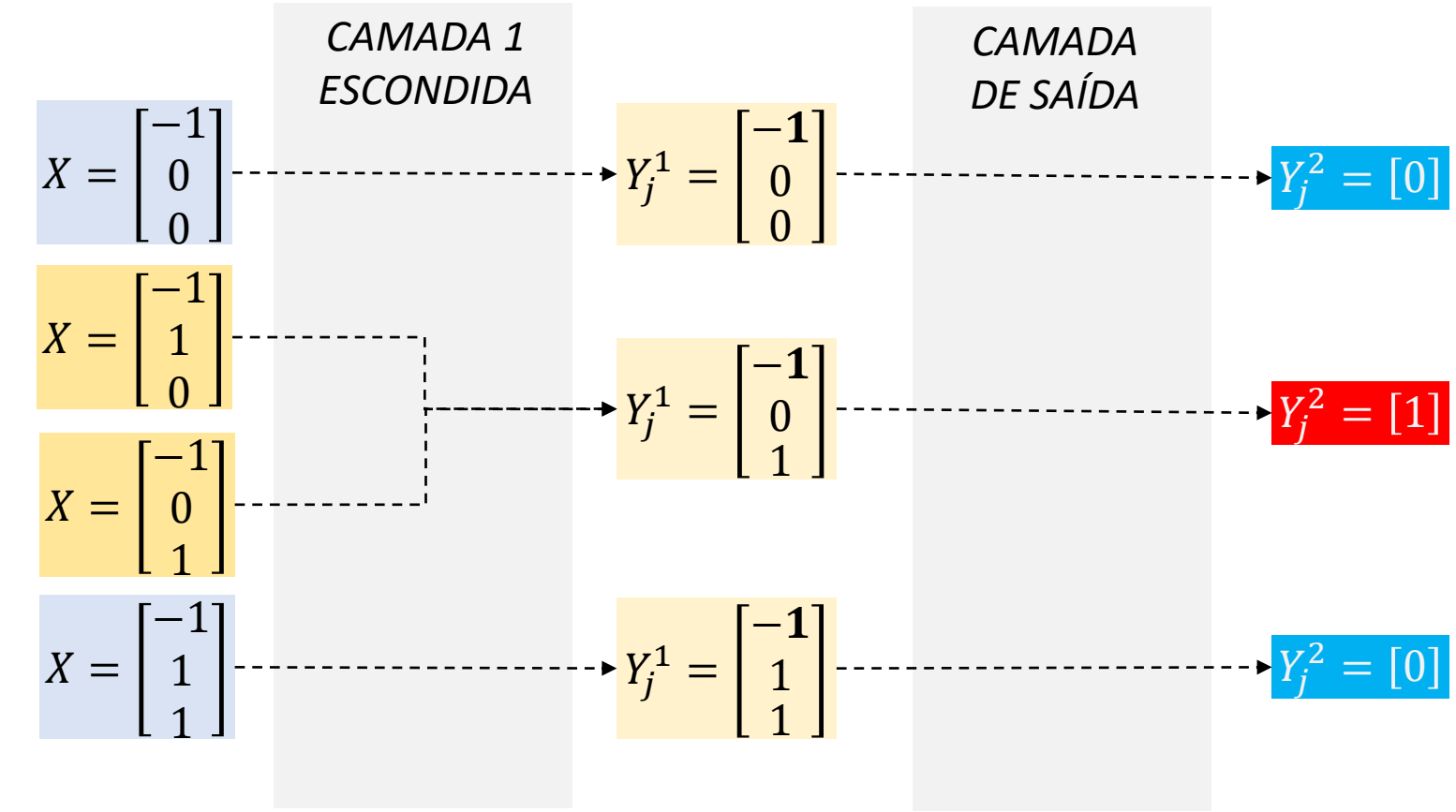
MLP: **Transformação espaço de entradas (XOR)**

RN XOR – feedforward para entradas
$x_1 = 1$ e $x_2 = 0$



As saídas dos neurônios da camada J são iguais quando a entrada é x_1=0 e x_2=1

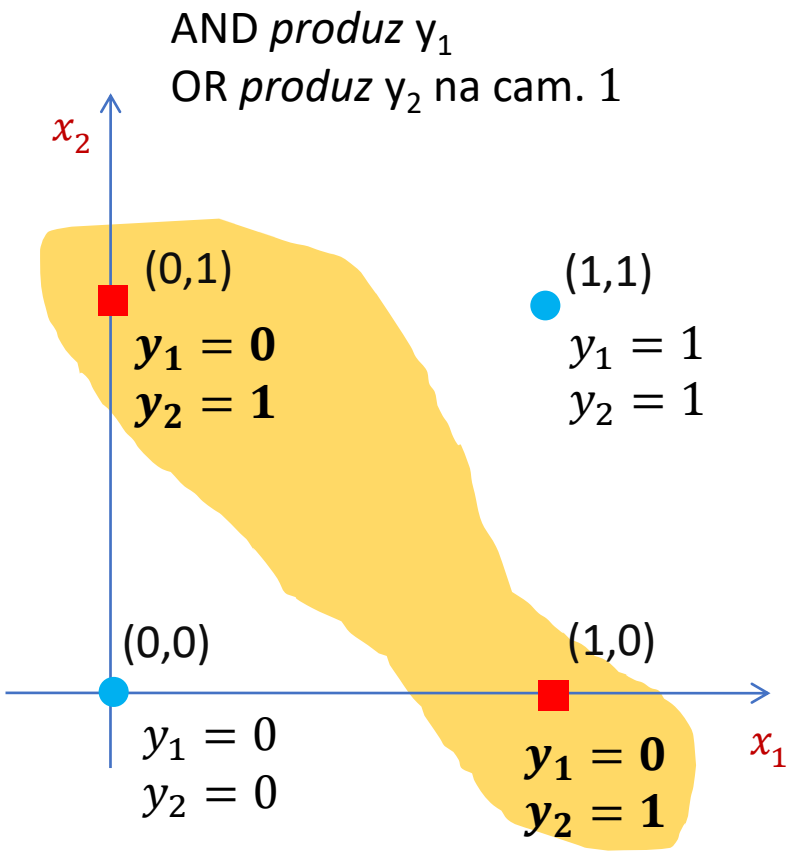
MLP: Transformação espaço de entradas (XOR)



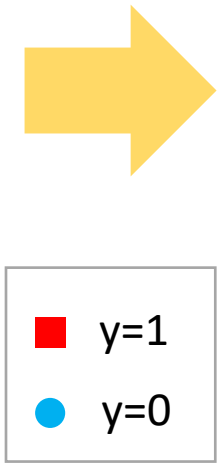
Transforma o espaço de entradas na camada escondida 1 reduzindo-o para a camada de saída

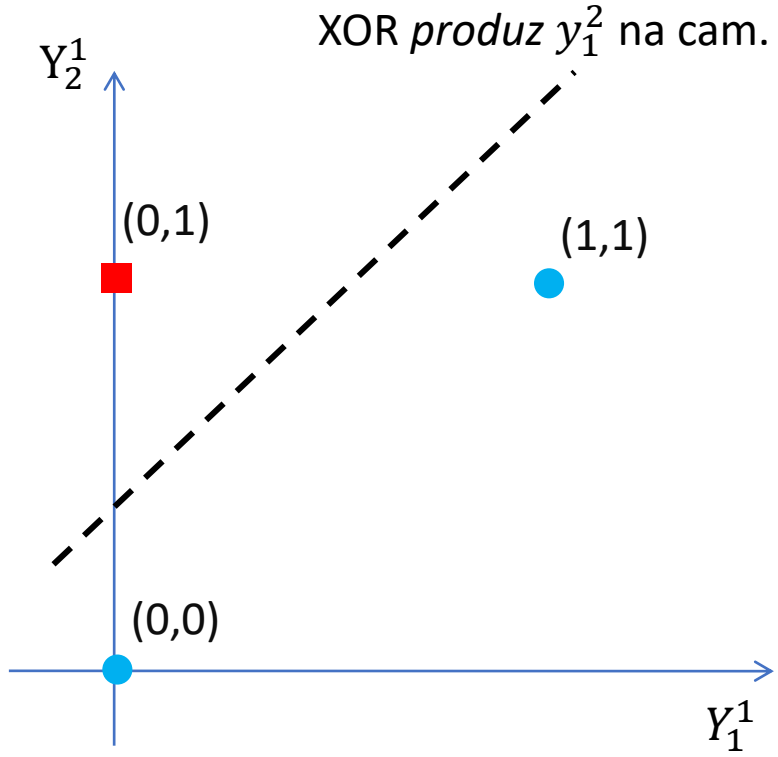
MLP: Papel da(s) camada(s) escondida(s)

Uma **camada escondida** transforma o espaço de entradas tornando-o linearmente separável.
Exemplo: XOR



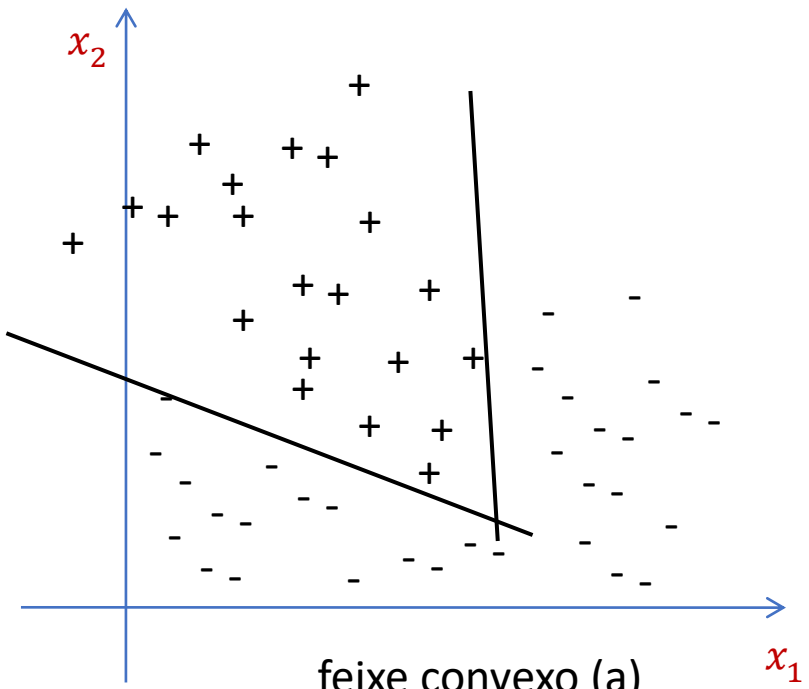
transformação do espaço de entradas



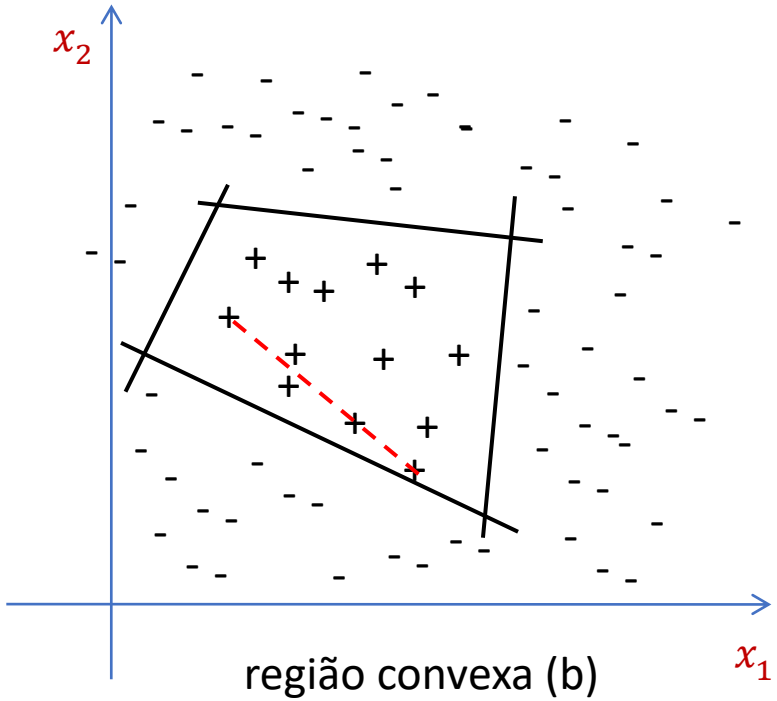


MLP: **Região convexa e não-convexa**

Uma MLP com **uma camada escondida** consegue separar apenas **regiões convexas (a) e (b)**.



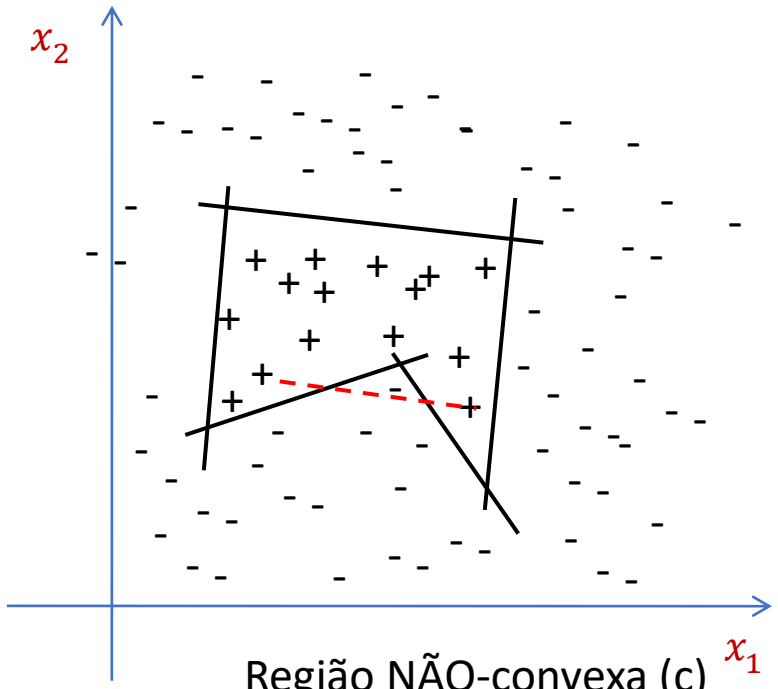
feixe convexo (a)



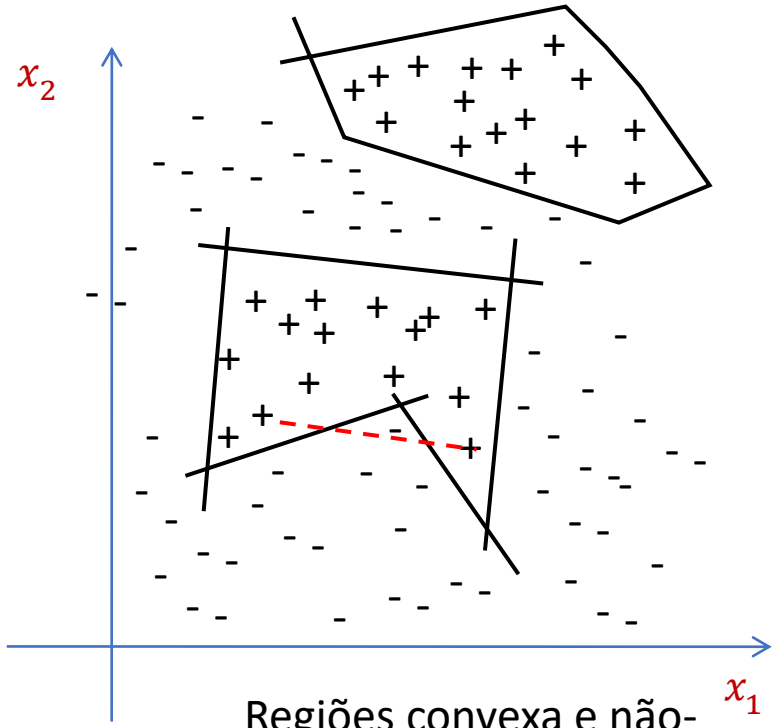
região convexa (b)

MLP: **Região convexa e não-convexa**

MLPs com **mais de uma camada escondida** são capazes de separar regiões não-convexas (c) e, inclusive, regiões disjuntas (d).



Região NÃO-convexa (c)



Regiões convexa e não-convexa disjuntas (d)

MLP: Exercício XOR

Exercício 1

* Defina outros pesos para a rede MLP para resolver o problema do XOR.
* Por que a separação não-linear se torna possível com este tipo de rede?

Funções de Ativação para Perceptrons

Outras funções além da degrau

PERCEPTRON **SIGMOIDE**

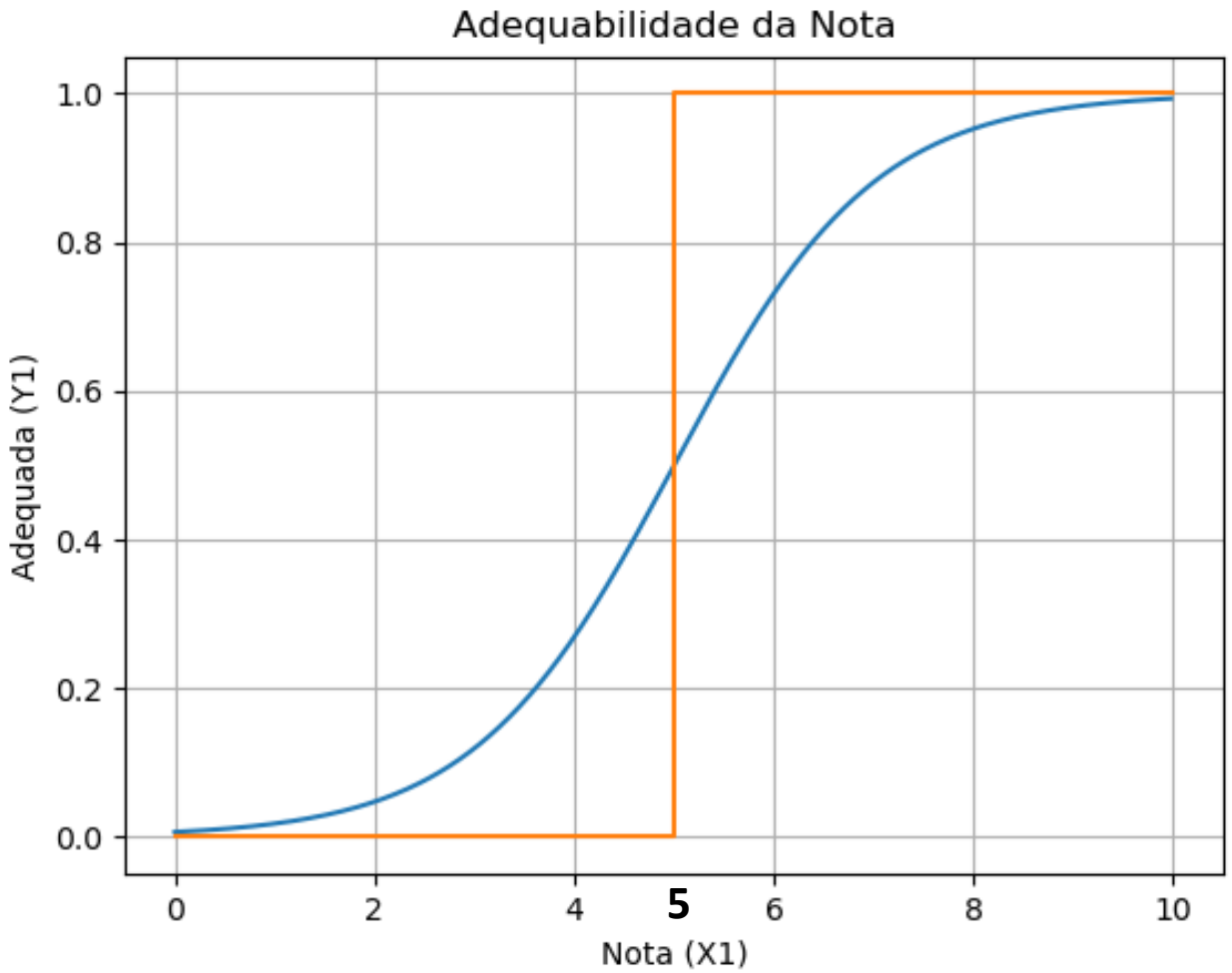
$$y_j = f(u_j) = \frac{1}{1 + e^{-u_j}}$$

w_0 = pto inflexão = 5
w = pesos (1)
x = entrada = nota
y = saída = adequada

Funções de ativação **sigmoides** permitem entradas e saídas no domínio dos reais. A saída é um real no intervalo [0, 1].

A **saída** de **um perceptron sigmoide** é suave (*e não binária como um perceptron com função degrau*).

Uma alteração pequena nos valores de entrada de um Perceptron **sigmoide** causa uma pequena alteração na saída (*não causa um mudança abrupta de 0 para 1 ou vice-versa – ver nota = X1= 5*)

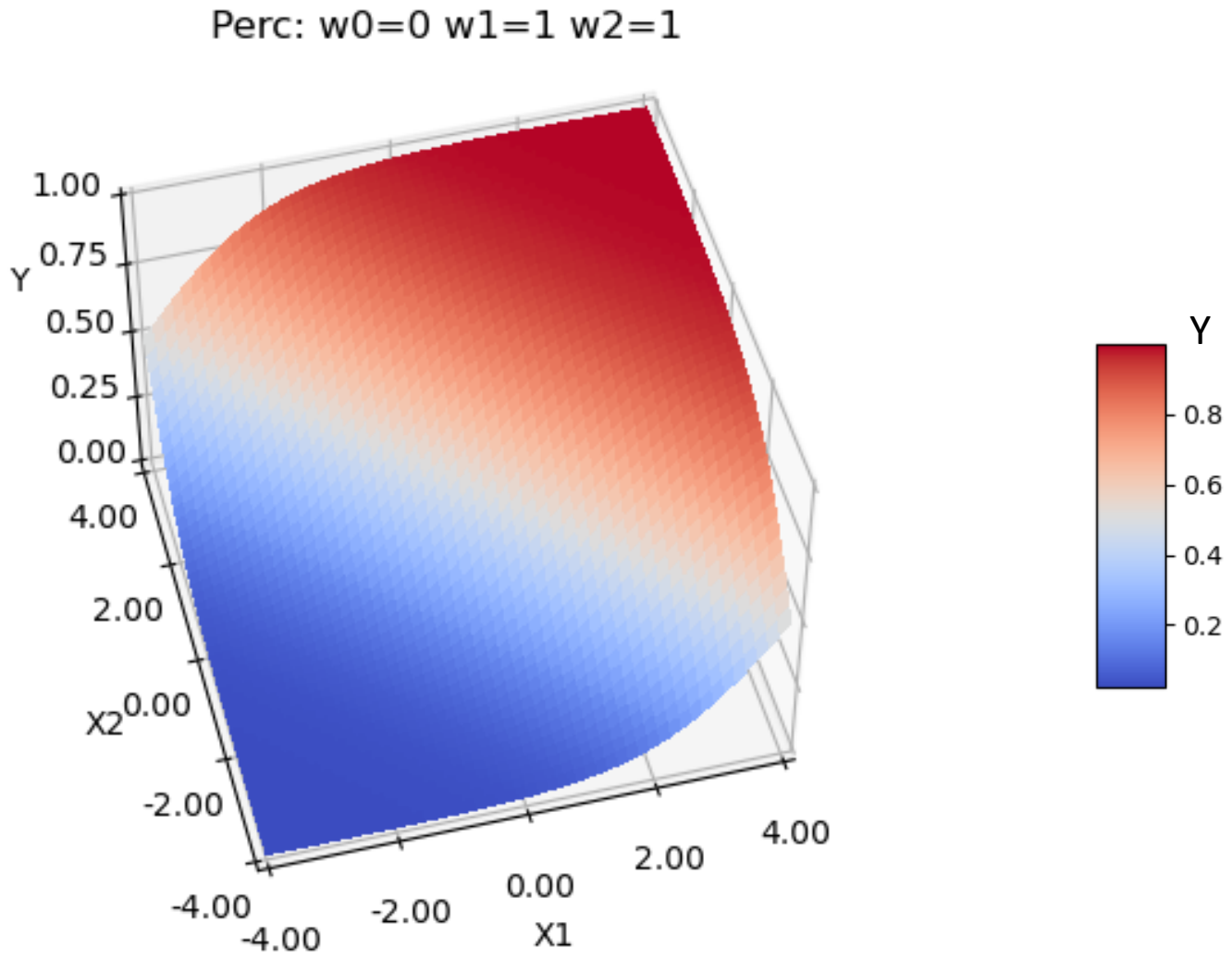


Perceptron sigmoide com duas entradas

$$y_j = f(u_j) = \frac{1}{1 + e^{-u_j}}$$

A saída do sigmoide é suave/graduada no intervalo [0,1]

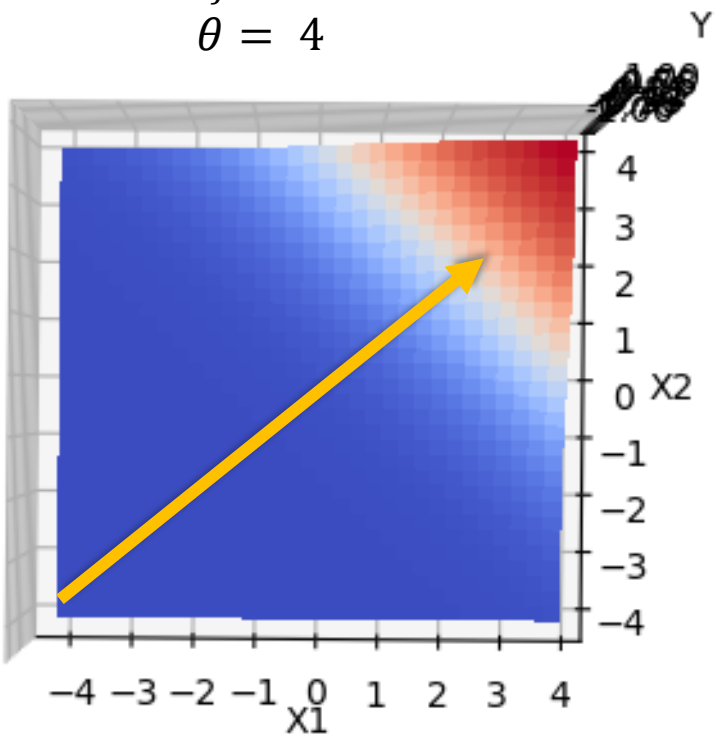
x_1	x_2	y_j
-4	-4	0.0
0	0	0.5
4	4	1.0



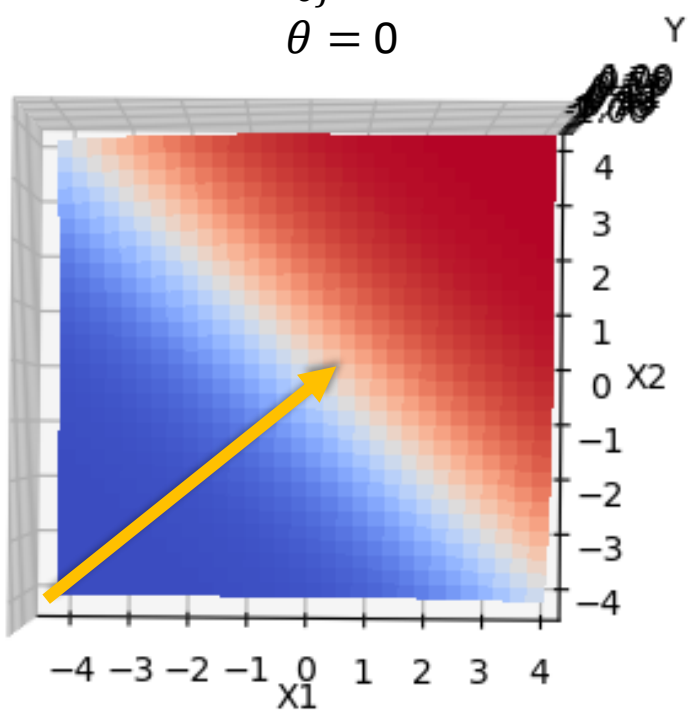
PERCEPTRON SIGMOIDE – **variações em w_{0j}**

$$w_{1j} = w_{2j} = 1$$

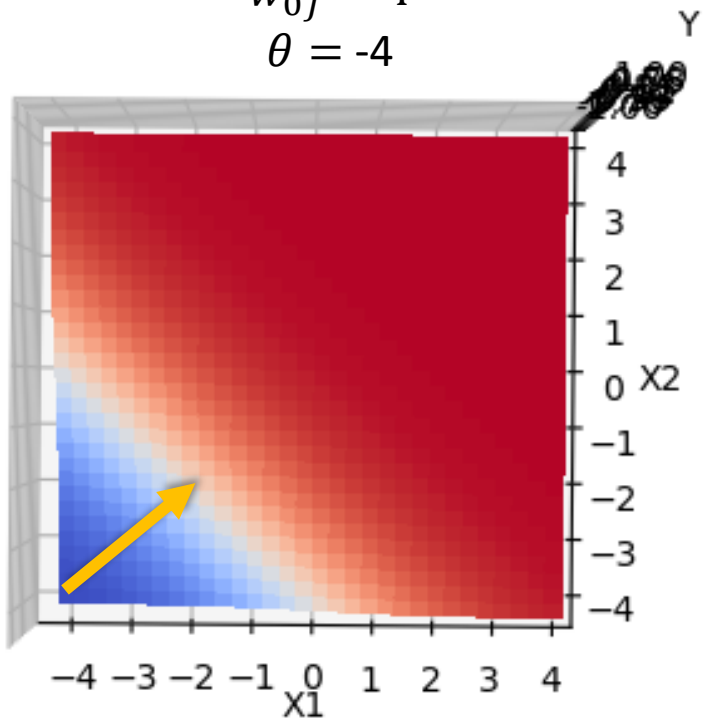
$w_{0j} = -4$
$\theta = 4$

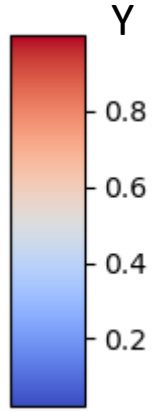


$w_{0j} = 0$
$\theta = 0$



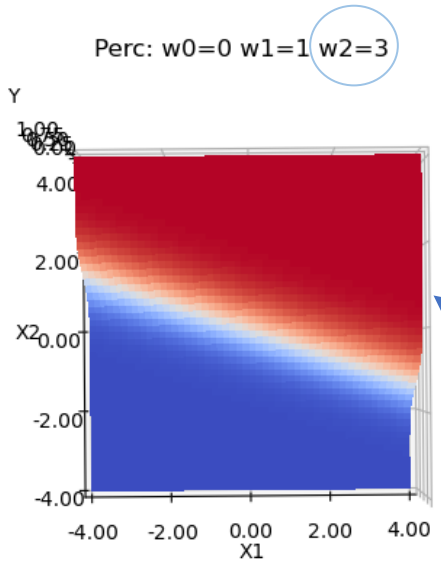
$w_{0j} = 4$
$\theta = -4$

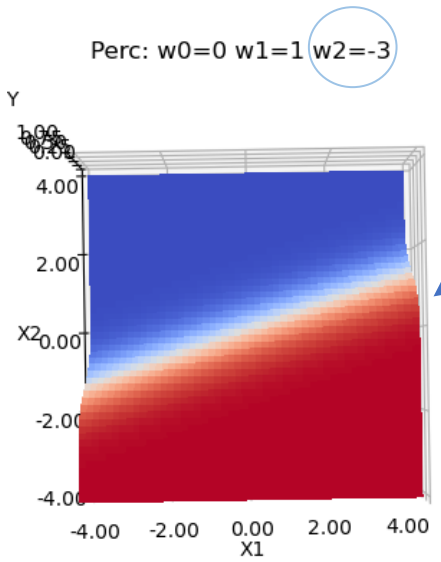


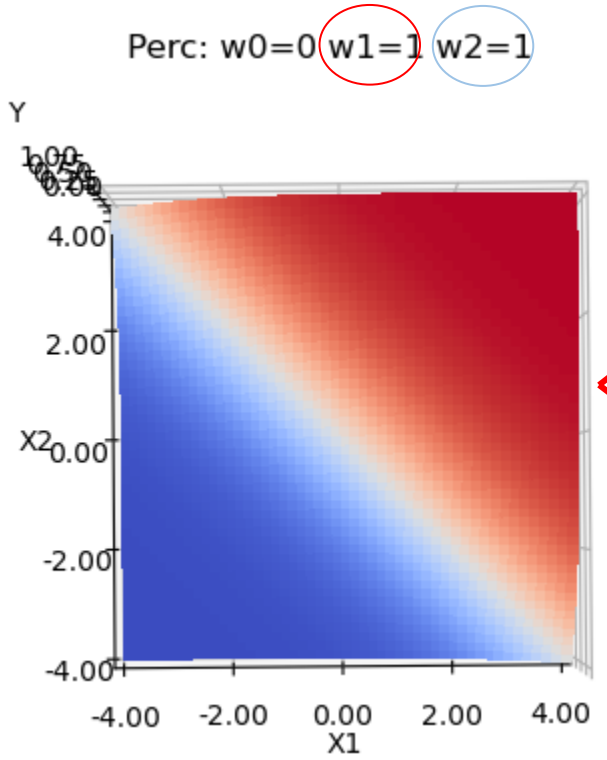


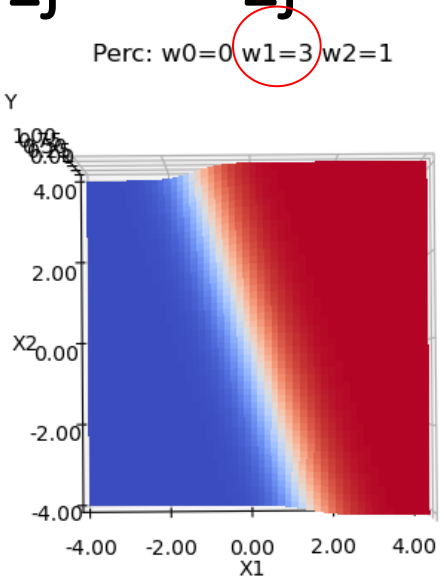
O ponto de inflexão diminui a medida que o w_0 aumenta (θ diminui)

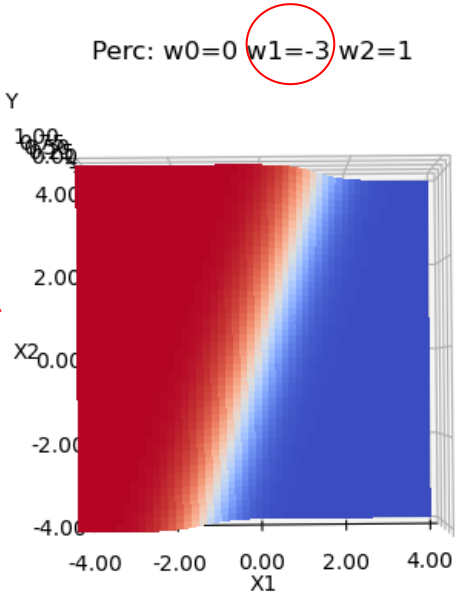
Perceptron sigmoide – **variações em w_{1j} e w_{2j}**







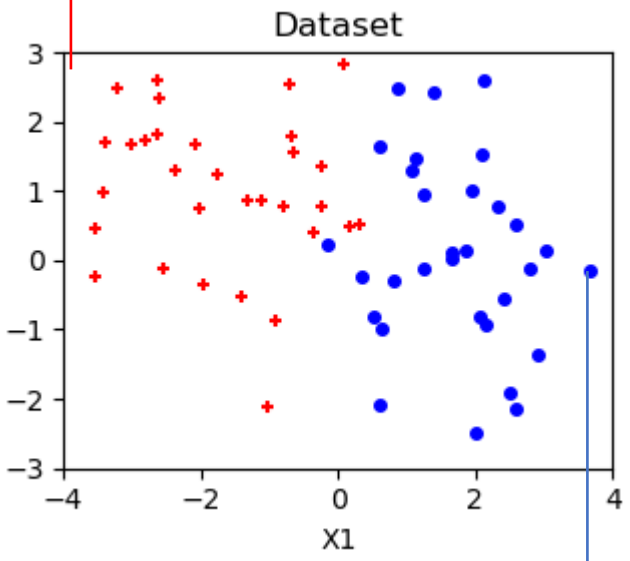




Perceptron sigmoide: **exemplo uso**

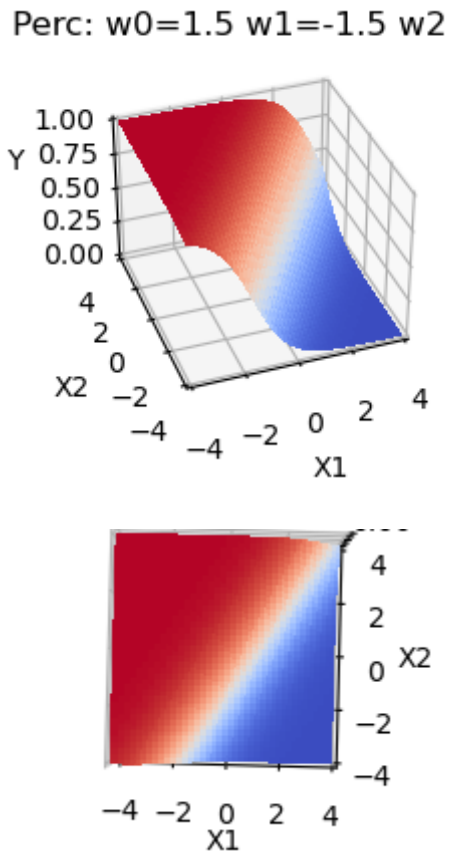
$$u_j = 1{,}5 - 4 * (-1{,}5) + 3 * 1 = 7{,}5$$

$$y_j = \frac{1}{1 + e^{-7{,}5}} = \mathbf{0{,}999}$$



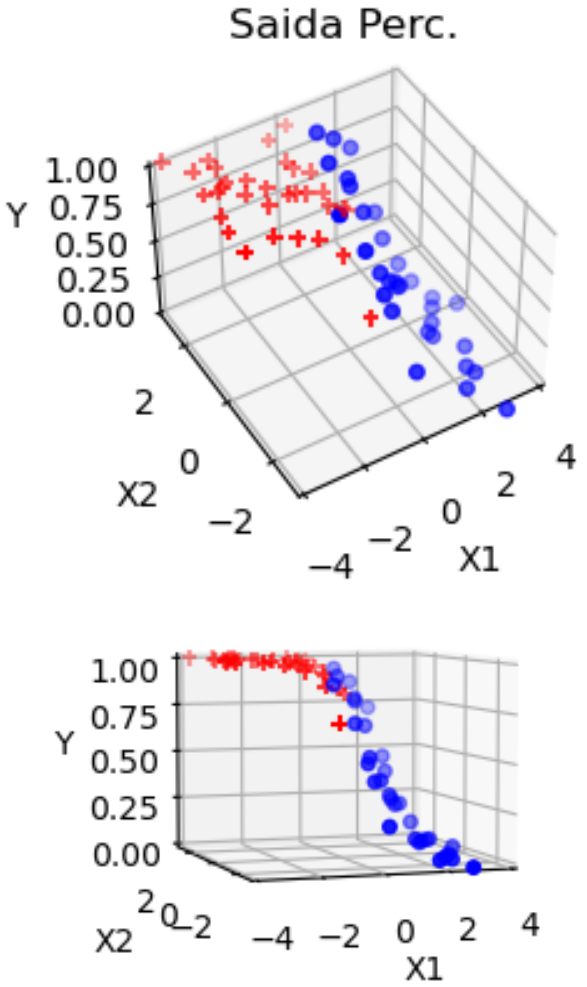
$$u_j = 1{,}5 + 4 * (-1{,}5) + 0 * 1 = -4{,}5$$

$$y_j = \frac{1}{1 + e^{4{,}5}} = \mathbf{0{,}010}$$



Perceptron sigmoide

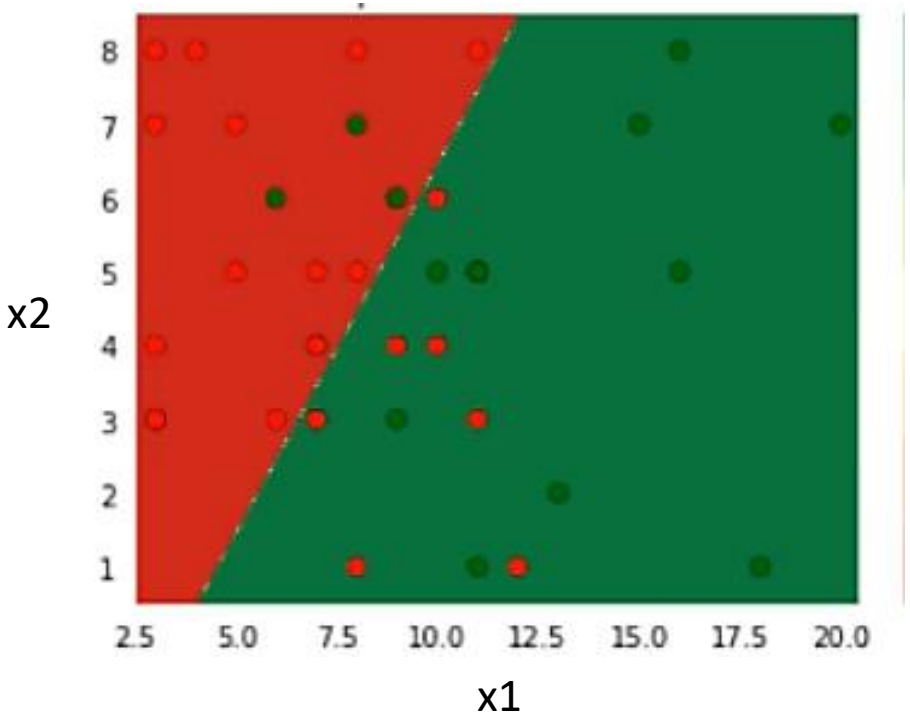
$$y_j = \frac{1}{1 + e^{-u_j}}$$



pontos de entrada são mapeados para um valor no intervalo [0, 1]

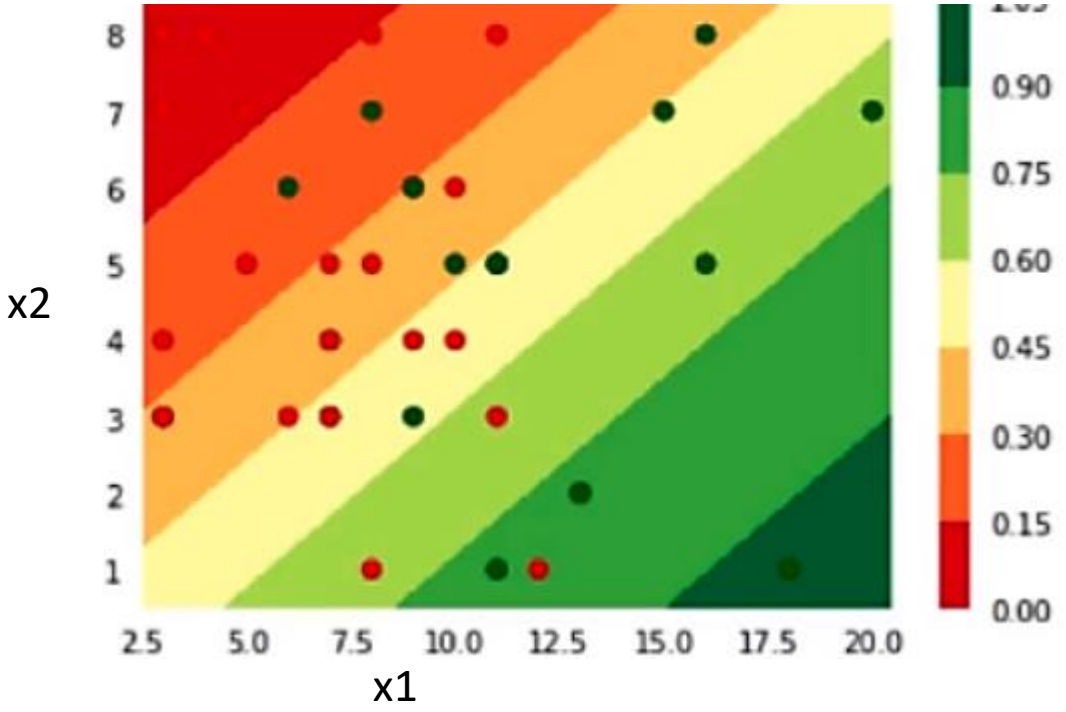
Perceptron: **degrau x sigmoide**

PERCEPTRON COM ATIVAÇÃO **DEGRAU**



Limites são rígidos

PERCEPTRON COM ATIVAÇÃO **SIGMOIDE**

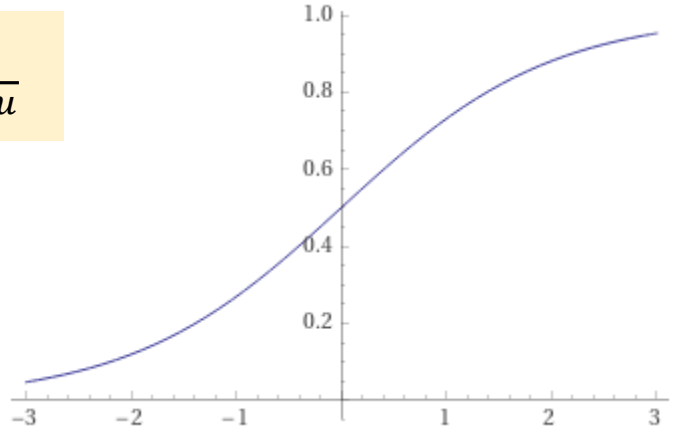


Ainda não separa, mas oferece limites mais suaves

Perceptron **Sigmoide** (sig)

$$y_j(u) = \text{sig}(u)$$

$$y_{j(u)} = \frac{1}{1 + e^{-u}}$$



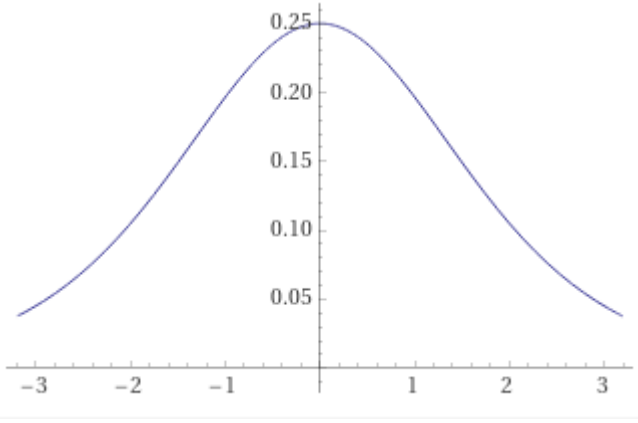
Uso
Se quiser uma saída entre [0, 1]

Transição de 0 para 1 é suave, mas rápida
– valores não muito distantes do ponto de inflexão são levados para 0 ou para 1
- (desvanecimento do gradiente)

Bastante usado em classificação binária

$$Derivada$$

$$\frac{e^u}{(1 + e^u)^2}$$



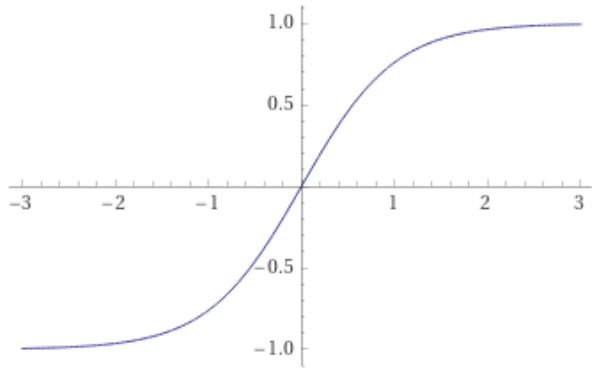
Desvantagens
- custo cálculo
- sofre de desvanecimento do gradiente para valores muito grandes ou muito pequenos da entrada

Perceptron **Tangente Hiperbólica (TanH)**

$$y_j(u) = \tanh(u)$$

$$y_j = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

$$y_j \in [-1, 1]$$

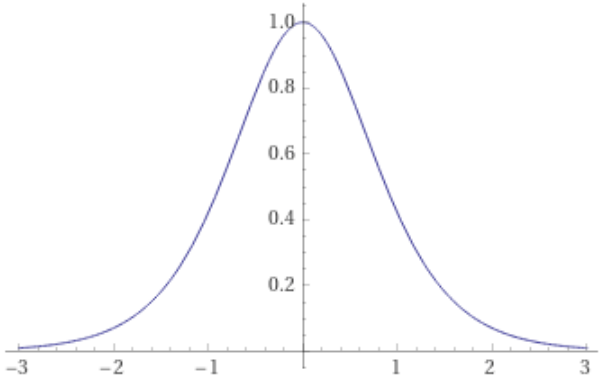


Uso

- <u>centrada no zero</u>
  (bom para características com valores negativos, neutros e positivos)
- valores de saída ficam entre [-1, 1]
- ajudam a 'centralizar' os dados nas camadas internas

$Derivada$

$$y'_j(u) = \frac{1}{\cosh^2(u)}$$



Desvantagens

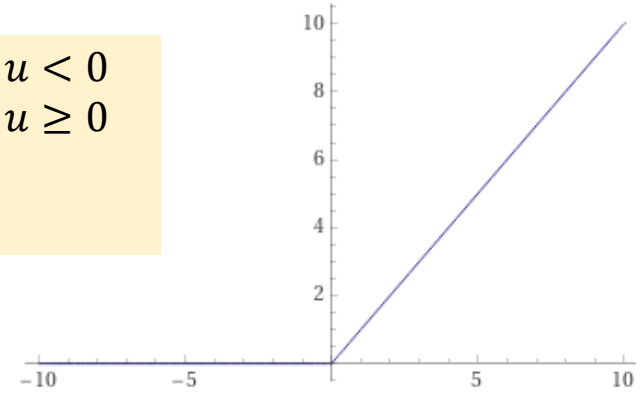
- custo cálculo
- sofre de desvanecimento do gradiente (nas beiradas)

PERCEPTRON **Rectified Linear Unit (ReLU)**

$$y_j(u) = relu(u)$$

$$y_j = \begin{cases} 0, & se\ u < 0 \\ u, & se\ u \geq 0 \end{cases}$$

$$y_j \in [0, \infty]$$



Uso

- evita e retifica o desvanecimento de gradiente (mas não completamente)
- muito usado em modelos de aprendizado profundo
- custo computacional mais baixo que sig, tanH
- utilizado nas camadas escondidas

$Derivada$

$$y_j'(u) = \begin{cases} 1, & u > 0 \\ 0, & u < 0 \\ indeterminado, & u = 0 \end{cases}$$

Desvantagens

- quando as entradas se aproximam de zero ou são negativas, o gradiente da função zera: a RN deixa de aprender
- sofre de desvanecimento do gradiente quando *u* se aproxima de zero (se igual a zero não tem derivada) ou quando u < 0 (derivada=0)

MLP propagação para frente

Feedforward

MLP: com perceptron sigmoides

Perceptron sigmoide: é uma unidade básica de construção de redes MLP que permite gerar separações não-lineares.

Por si só, não é capaz de criar separações não-lineares, mas ajudará!

O modelo aprendido resulta da ligação ponderada **de vários neurônios sigmoides** de diferentes camadas que *se aproxima de uma função desconhecida* definindo assim limites não-lineares de separação das saídas

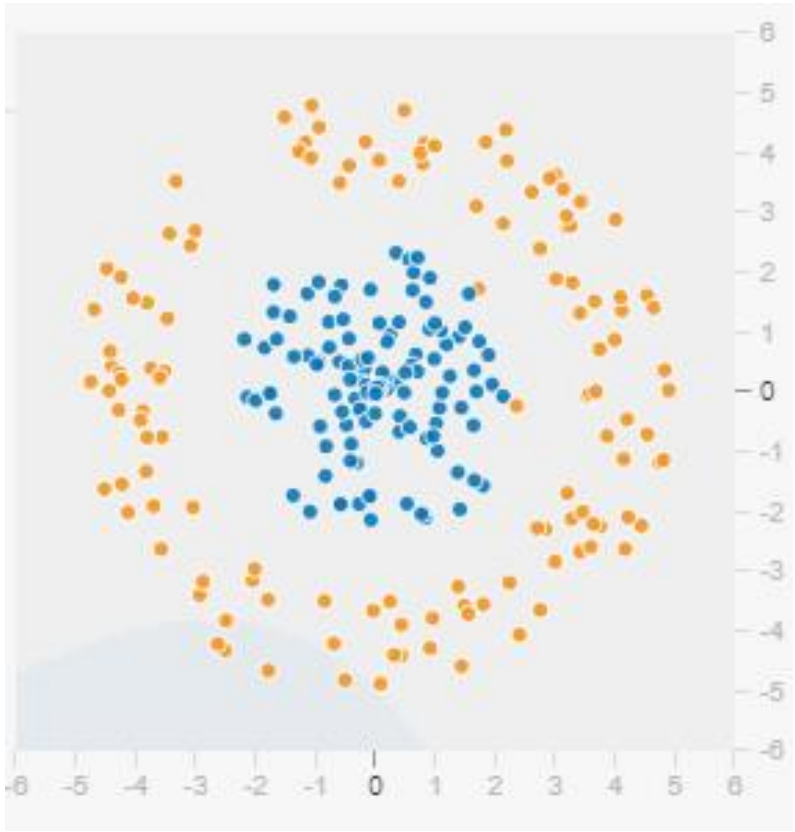
MLP: Multilayer perceptron

MLP: **Exemplo Classificação Binária**

PROBLEMA

Queremos predizer se algo pertence a classe dos azuis ou laranjas baseando-se na relação de duas *features*:

$$X1 \times X2$$

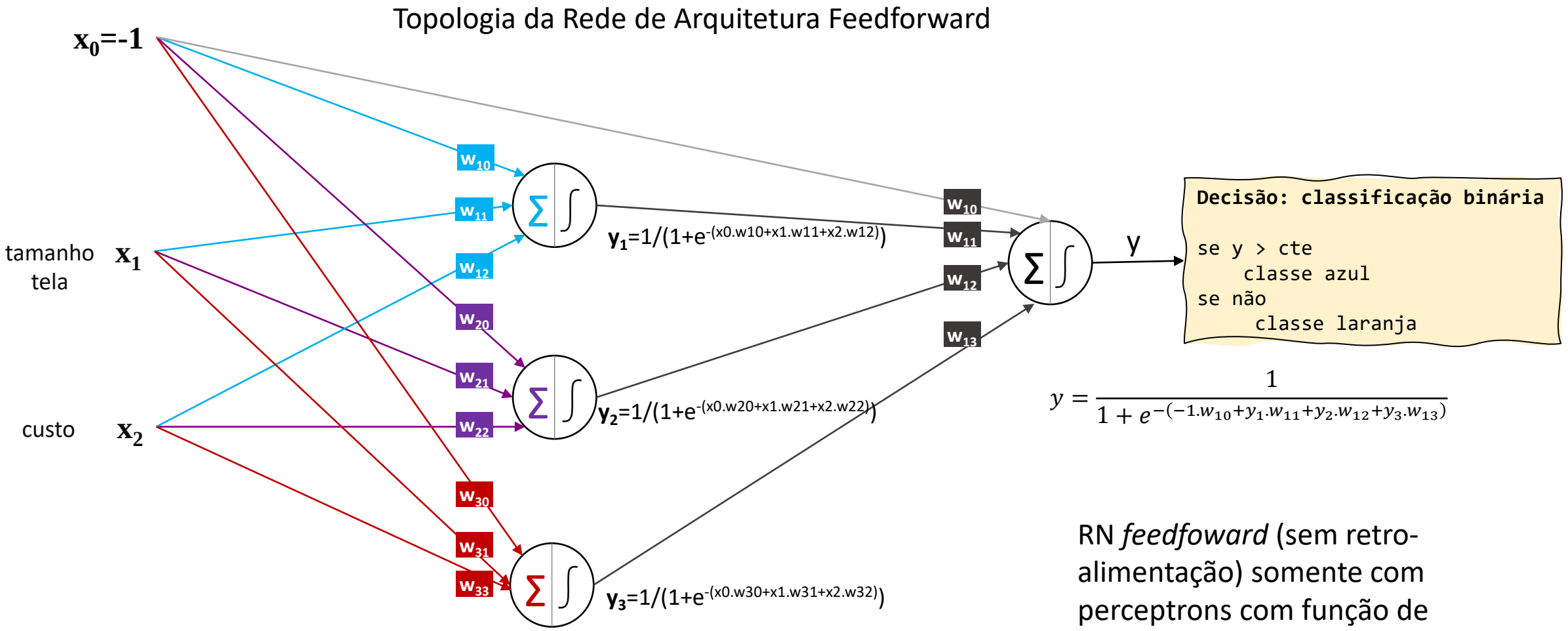


X2

X1

[fonte: tensorflow playground]

MLP: Exemplo Classificação Binária



RN *feedfoward* (sem retro-alimentação) somente com perceptrons com função de ativação sigmoide.

**Sobrescritos indexadores das camadas foram omitidos*

MLP: **Exemplo Classificação Binária**

O objetivo é encontrar os pesos para os neurônios de todas as camadas da rede de forma que a superfície em amarelo (saída) valha próximo de 1 para os pontos vermelhos e próximo de zero para os azuis.

MLP: exemplo 2 – **classificação binária**

PESOS

Camada escondida 1

$$W_{ji}^{1} = \begin{bmatrix} -4{,}2 & -2{,}3 & -0{,}76 \\ -3{,}6 & 1{,}3 & -1{,}2 \\ -3{,}1 & 0{,}39 & 1{,}6 \end{bmatrix}$$

j
Neur. 1
Neur. 2
Neur. 3

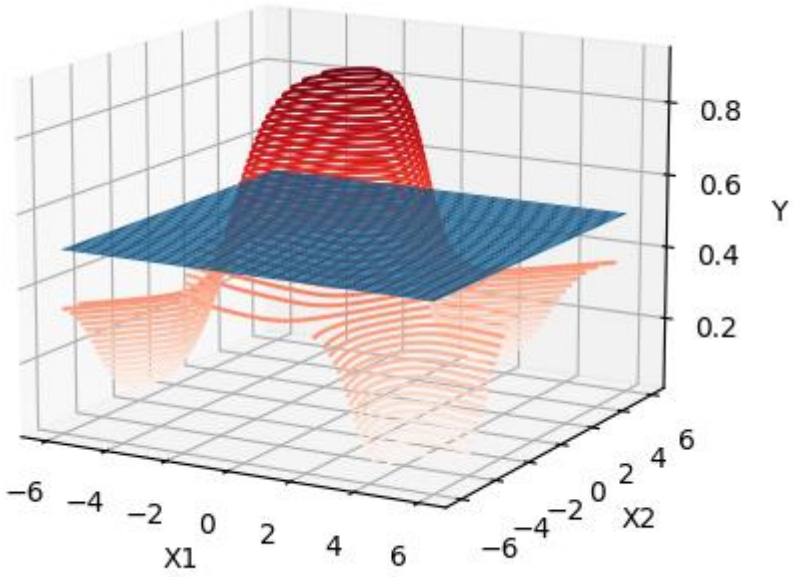
Camada de saída

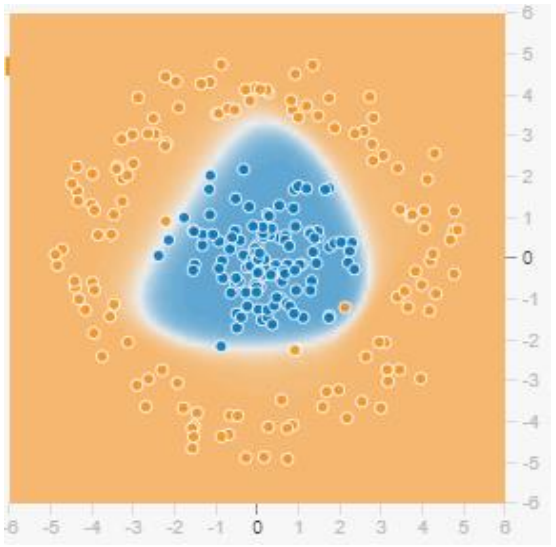
$$W_{ji}^{2} = [7{,}2 \quad 3{,}4 \quad 3{,}3 \quad 3{,}4]$$

Neur. 1

Decisão saída
Plano em z = 0,5

Vistas da saída
da rede neural





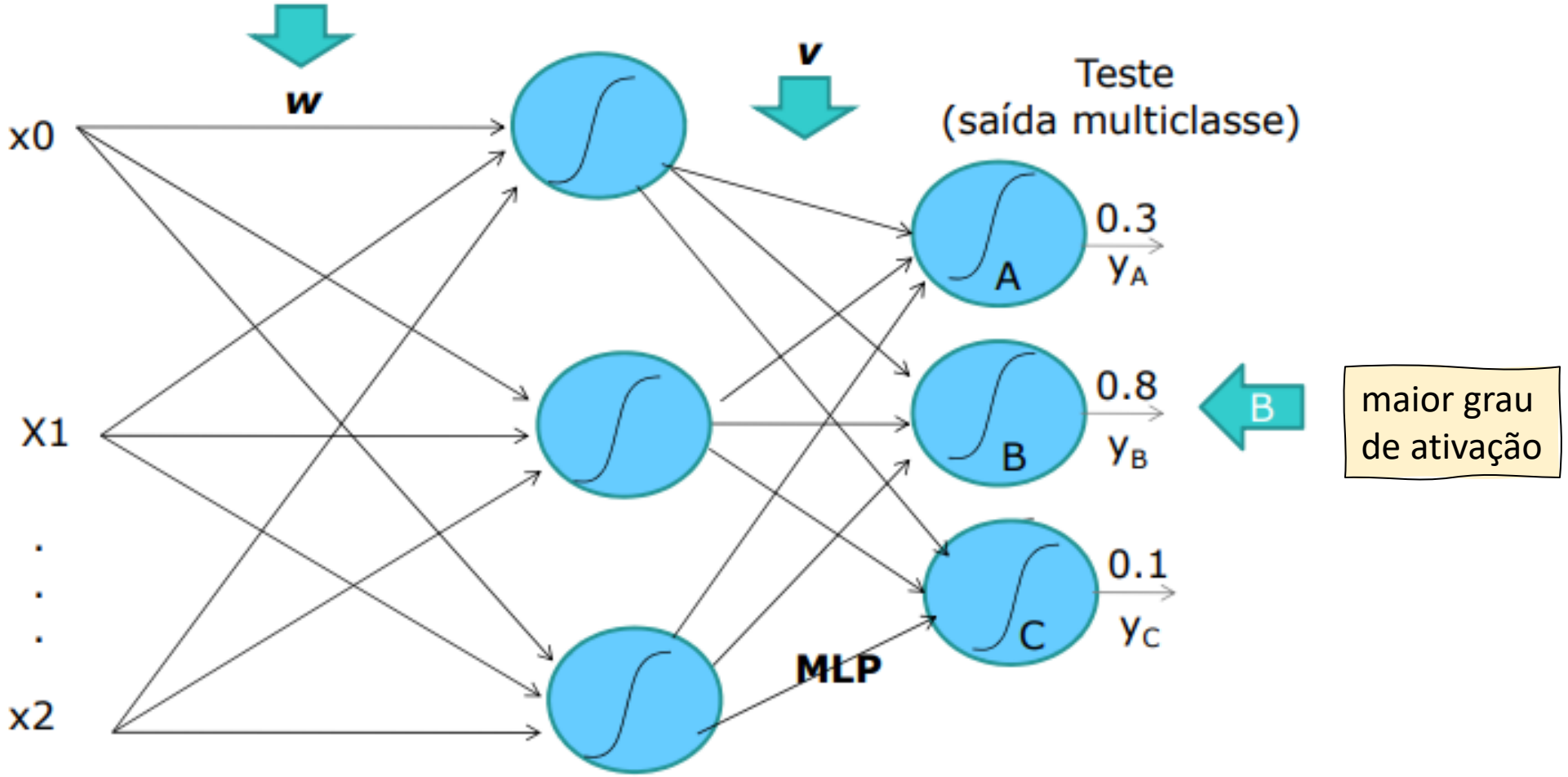
X2

X1

[fonte: tensorflow playground]

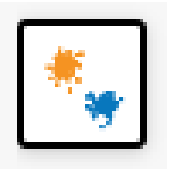
código disponível em [plot_RN_MLP231_sig]

MLP: e se a classificação for para n classes?



Preparação para treinamento...

No playground.tensorflow.org



Dataset
- selecione o dataset linearmente separável (laranjas = - e azuis = +)
- ratio training/test data: 70% (clique em show test data)
- Noise: 0
- batch size: 1

Topologia da rede
- escolha as features X_1 e X_2
- coloque 1 camada escondida com 1 neurônio

Hiperparâmetros:
- learning rate: 0.03
- activation function: sigmoid
  regularization: none
- regularization rate: 0
- problem type: classification

Observe:
- número de épocas para estabilizar o erro treinamento (training loss) e validação (test loss)

- a separação linear entre os exemplos azuis e laranjados

Faça:
- aumente o ruído (noise) misturando os azuis e laranjas
- coloque uma tx de aprendizado bem pequena e observe o número de épocas para aprendizado
- coloque uma tx de aprendizado grande (3) e observe a oscilação

Preparação para treinamento...

No playground.tensorflow.org

Dataset
- selecione o dataset não-linearmente separável 
- ratio training/test data: 70% (clique em show test data)
- Noise: 0
- batch size: 1

Topologia da rede
- escolha as features X_1 e X_2
- coloque 1 camada escondida com 1 neurônio

Hiperparâmetros:
- learning rate: 0.03
- activation function: **sigmoid**
  regularization: none
- regularization rate: 0
- problem type: classification

Observe:
- o erro é grande porque um neurônio sigmoide não é capaz de separar dados que estão dispostos de forma não-linear

- insira mais um neurônio sigmoide na camada escondida 1 (também não resolve)

- insira mais uma camada escondida com dois neurônios e deixe a camada de saída com 2 neurônios

- uma solução é: 2, 3, 1 (neurônios por camada – da entrada até a saída)

Treinamento de MLP

TREINAMENTO/APRENDIZADO

Aprendizado Conexionista

processo no qual os **parâmetros livres** de uma Rede Neural Artificial (RNA) são alterados pela estimulação contínua causada pelo ambiente no qual a rede está inserida, da seguinte forma:

ESTÍMULO → ADAPTAÇÃO → NOVO COMPORTAMENTO

É um processo iterativo de **ajuste dos parâmetros** da rede. Pode ser supervisionado, não-supervisionado e por reforço.

MLP: Treinamento supervisionado

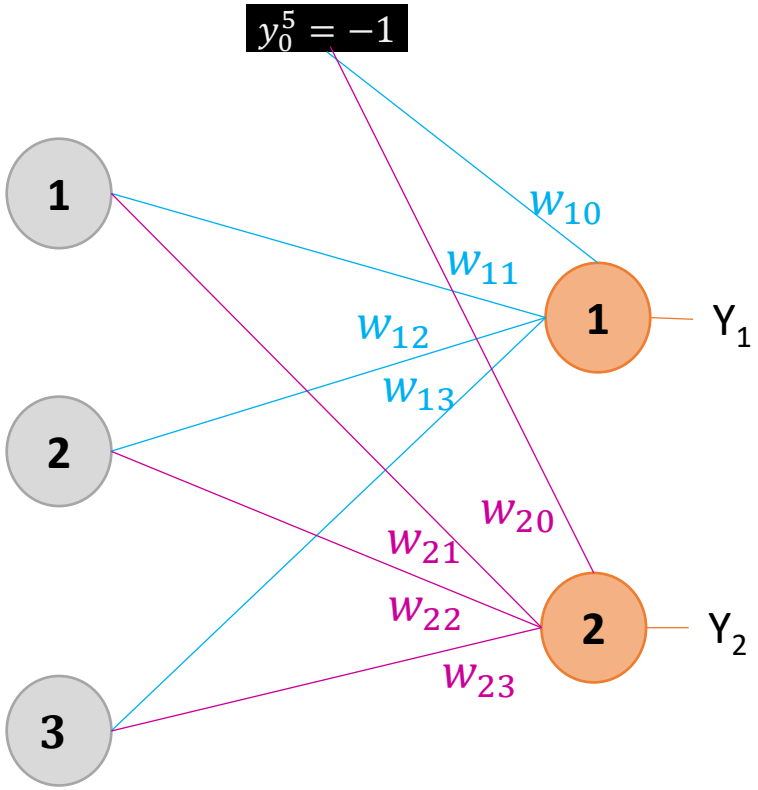
No aprendizado supervisionado, o problema de treinamento consiste em **ajustar os pesos** da RN para **minimizar globalmente** o **erro do modelo** calculado para um dado conjunto de dados de entrada.

MLP: Erro quadrático de treinamento

O erro de predição para cada padrão/exemplo ***k*** de treinamento pode ser calculado pelo **erro quadrático** entre a saída esperada ***d*** e a obtida ***y*** pelo neurônio *j.*

O erro total é a somatória dos erros de cada neurônio da camada de saída.

MLP: Erro quadrático de treinamento para os neurônios de saída



$d_j(k)$ = saída desejada (esperada) para o exemplo k

$Y_j(k)$ = saída produzida pelo neurônio de saída j para o exemplo k

$$E(K) = \frac{1}{2}.\left(d_1(k) - Y_1^5(k)\right)^2$$

$$E(K) = \frac{1}{2}.\left(d_2(k) - Y_2^5(k)\right)^2$$

$$E(k) = \frac{1}{2}.\sum_{j=1}^{n_L}\left(d_j(k) - y_j^L(k)\right)^2$$

Erro quadrático para um exemplo K na camada de saída = L (Eq. 1)

$\mathbf{n_L}$ = número de neurônios na camada de saída

... escondida 4 saída 5

MLP: Erro quadrático médio para uma época

ρ= número de exemplos de treinamento

$$E_M = \frac{1}{\rho} . \sum_{k=1}^{n_\rho} E(K)$$

Erro quadrático médio de treinamento (Eq. 2)

MLP: **Algoritmo backpropagation**

Essência do algoritmo: fazer ajuste dos pesos calculando a influência de cada um dos pesos (inclusive do limiar ou bias) de toda a RN no erro produzido na saída.

$$\frac{\partial E}{\partial W}$$

- ∂E — erro de saída
- ∂W — vetor de pesos

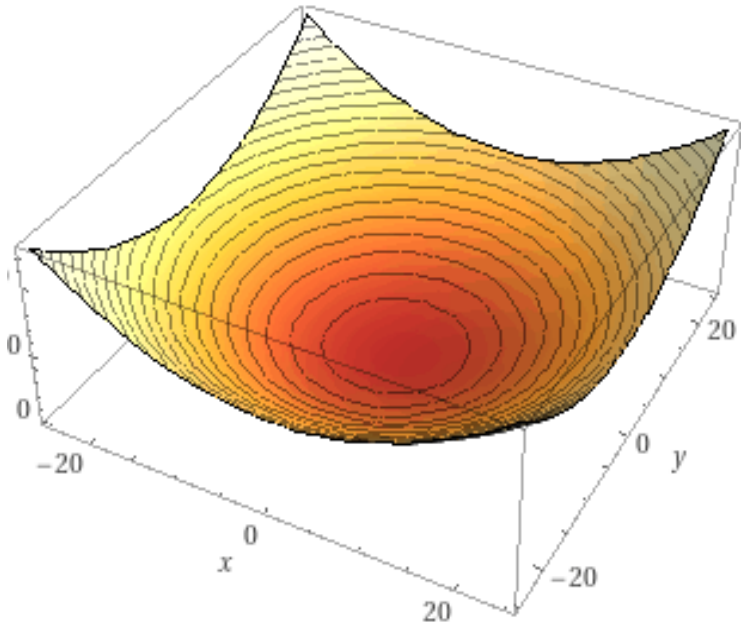
Utiliza descida de gradiente para ajustar os pesos (aumentar ou diminuir)

Rumelhart, D., Hinton, G. & Williams, R. Learning representations by back-propagating errors. *Nature* **323,** 533–536 (1986). https://doi.org/10.1038/323533a0

Algoritmo backpropagation: **revisão descida de gradiente**

Dada uma função multivariável (de m variáveis), o gradiente é um vetor de $m\ linhas \times 1\ coluna$ contendo todas as derivadas parciais da função.

$$f(x, y) = x^2 + y^2$$



$$\frac{\partial f}{\partial x} = 2x$$

$$\frac{\partial f}{\partial y} = 2\mathrm{y}$$

derivadas parciais de $f(x, y)$

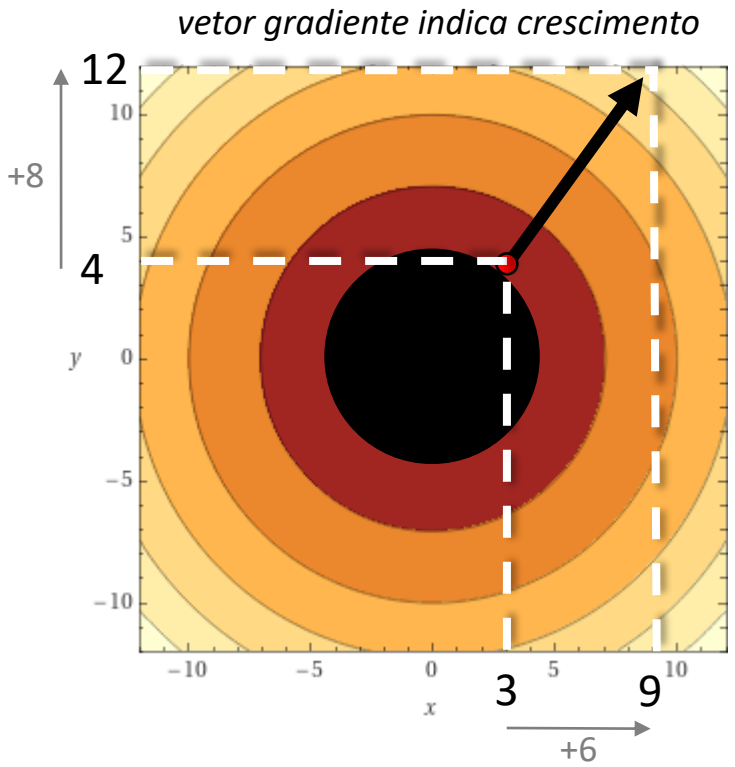
$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$$

vetor gradiente de $f(x, y)$

$\nabla =$ *'nabla', 'del'*

Algoritmo backpropagation: **revisão descida de gradiente**

Como a função se comporta em relação à direção do crescimento no ponto (3,4)?



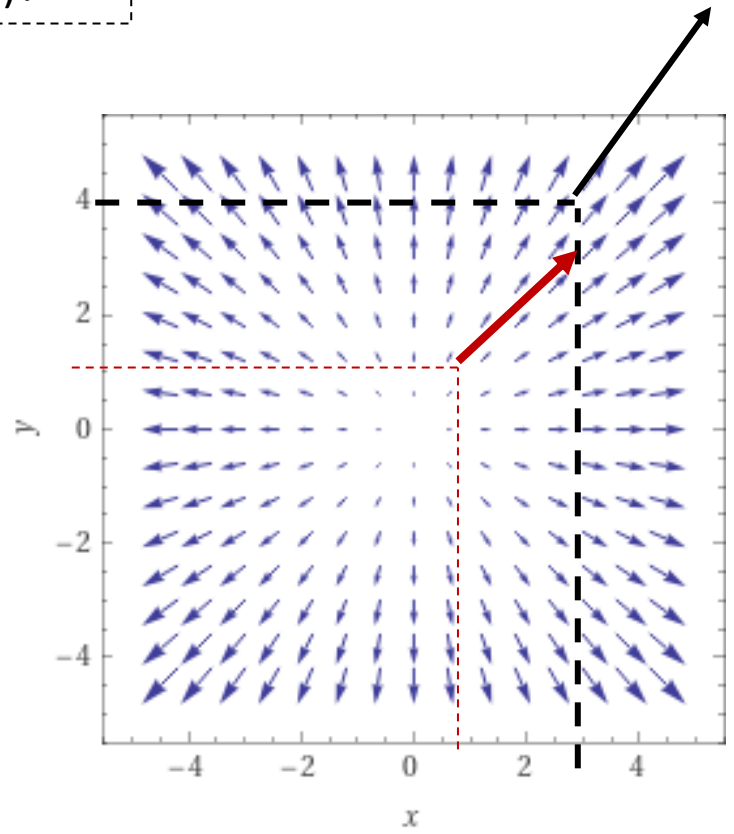
contornos para
$f(x,y) = x^2 + y^2$

$$\nabla f(x,y) = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$$

Vetores grad.

$$\nabla f(3,4) = \begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

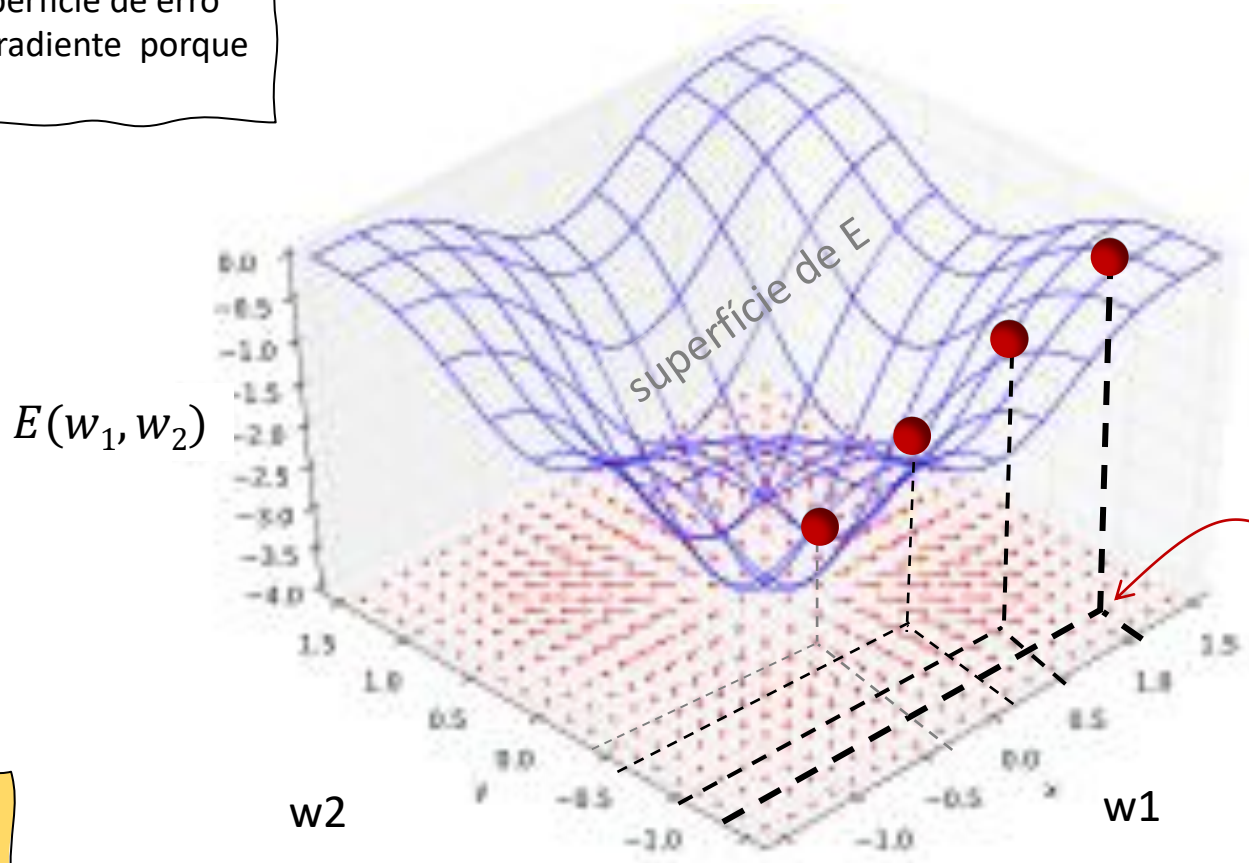
$$\nabla f(1,1) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$



vetores gradiente para vários pontos

MLP: Descida de gradiente (visão intuitiva)

Descida de gradiente : Cada treinamento corresponde a um percurso na superfície de erro na direção contrária dos vetores gradiente porque queremos minimizar o erro



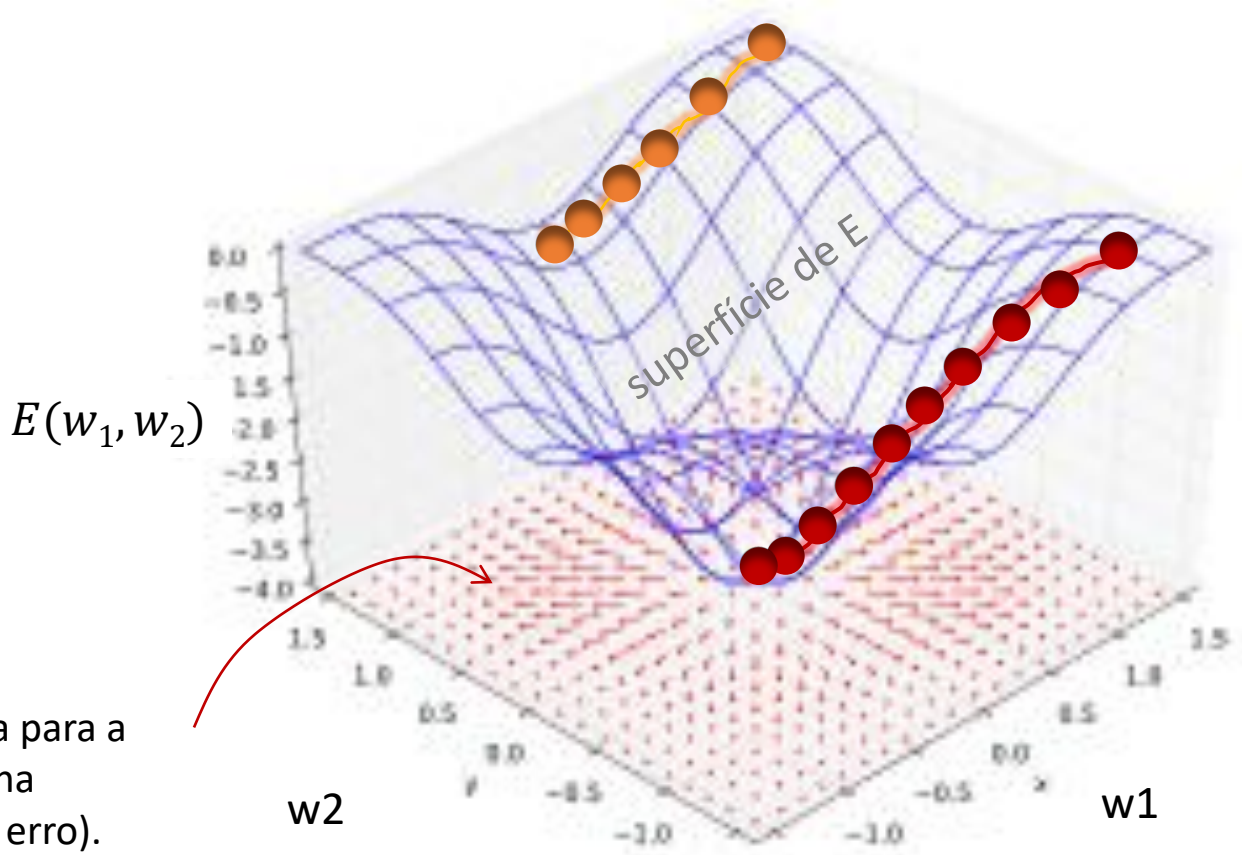
$E(w_1, w_2)$

Vetor gradiente da função de erro *– dado um par (w1, w2) resulta em um vetor que indica o crescimento mais íngreme do erro no ponto. A ideia é ir na direção contrária!*

$$\nabla E(w_1, w_2) = \begin{bmatrix} \frac{\partial E}{\partial w_1} \\ \frac{\partial E}{\partial w_2} \end{bmatrix}$$

A descida pode parar em um mínimo local ou no mínimo global

MLP: Descida de gradiente (visão intuitiva)



Gradiente da função: aponta para a direção de maior inclinação na ascendente (crescimento do erro).

MLP: Algoritmo backpropagation

Estratégia:

para cada exemplo

- faz feedforward das entradas produzindo as saídas sem alterar os pesos
- para cada neurônio de saída, calcular o erro em relação ao valor desejado
- fazer backpropagation do erro para ajustar os pesos dos neurônios de cada uma das camadas (*da camada de saída em direção à camada de entrada)* alterando-o levemente com um valor proporcional a derivada parcial do erro em relação a cada peso

Critério de parada

o erro é pequeno o suficiente **ou**
o erro fica estagnado após várias épocas **ou**
atingimos um número fixo de épocas

MLP: Pseudocódigo do backpropagation

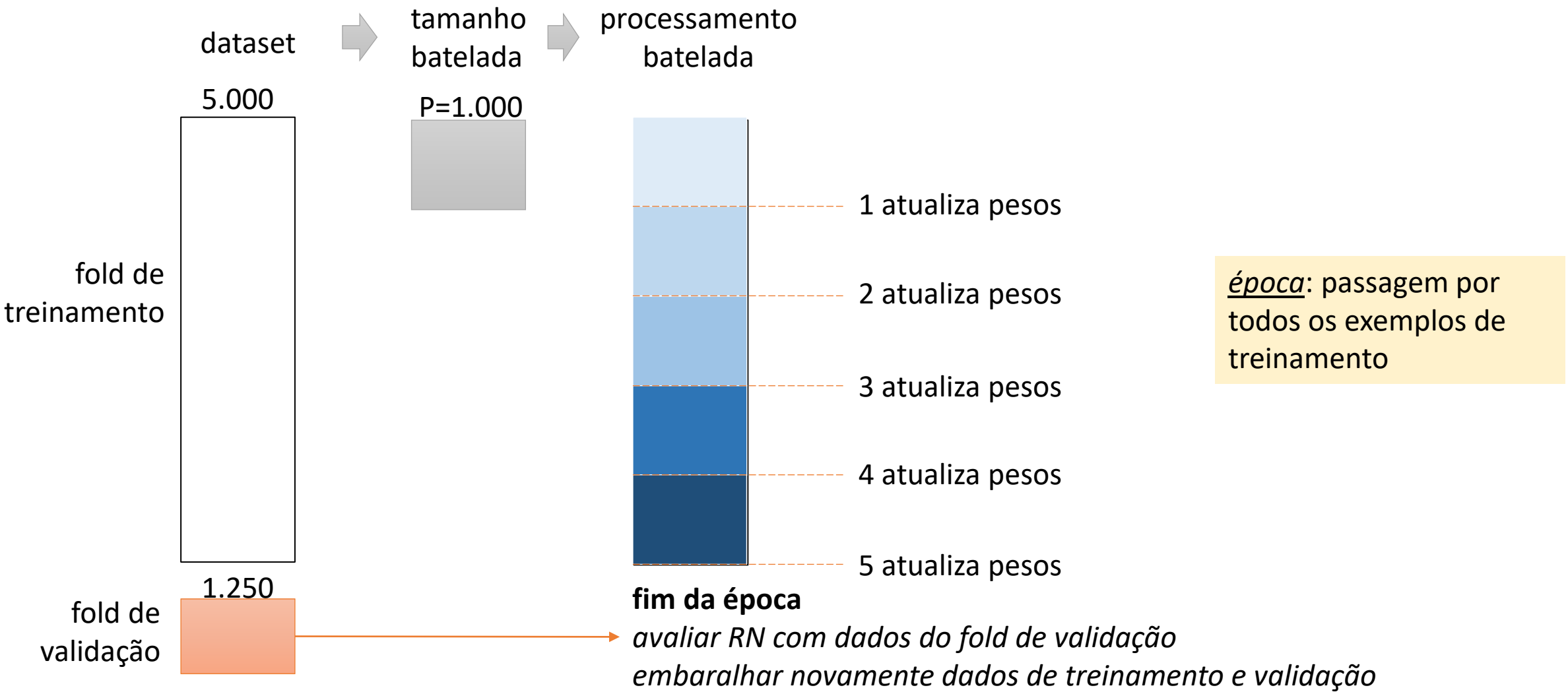
```
Inicializar pesos W da rede com valores aleatórios [-1, 1]
η:= 0.03 (inicializar taxa de aprendizado com valor pequeno [0.001, 0.05]
ϵ := 0.02 (erro mínimo almejado)
E_Mant := (erro quadrático médio anterior) inicializar com valor alto
E_Matu := (erro quadrático atual) inicializar de forma que | E_Mant - E_Matu | > ϵ
época := 1
// treinamento
enquanto epoca < MÁX e | E_Mant - E_Matu | > ϵ
    para cada exemplo k do dataset de treinamento:
        // Passo feedforward
        Calcular saídas da RN: Y_j

        // Passo backpropagation – atualizar pesos
        Calcular o erro de treinamento E(k)
        Calcular o ajuste ΔW_ji para cada camada da RN e atualizar cada um dos pesos:
        W_ji := W_ji − ηΔW_ji  (tam. batelada = 1)

    // validação – avalia a generalização da rede atual
    E_Matu := calcular erro usando a RN com os novos pesos utilizando os dados de validação
    Separar novamente dados de treinamento dos dados de validação
    época += 1
```

Δ**w**: alteração de todos os pesos frente o processamento do padrão k (-1 porque vai na direção contrária ao do crescimento do gradiente); alteração dos pesos para cada exemplo (tam. batelada = 1)

MLP: **Backpropagation – batelada e K-folds**



época: passagem por todos os exemplos de treinamento

MLP: Backpropagation – atualização dos pesos

instante da atualização

Por padrão: pesos são recalculados e alterados após apresentação de cada padrão/instância. A próxima instância já é treinada com os novos pesos.

Obs: a ordem de apresentação deve ser modificada a cada época

Ajuste a cada padrão:

$\mathbf{w}(\boldsymbol{i}) \coloneqq \boldsymbol{w}(i-1) - \boldsymbol{\alpha}\Delta w$

(i é o número da iteração)

Por batelada: pesos são recalculados a cada apresentação de padrão, mas a alteração é feita após o processamento de toda batelada

Ajuste a cada P padrões

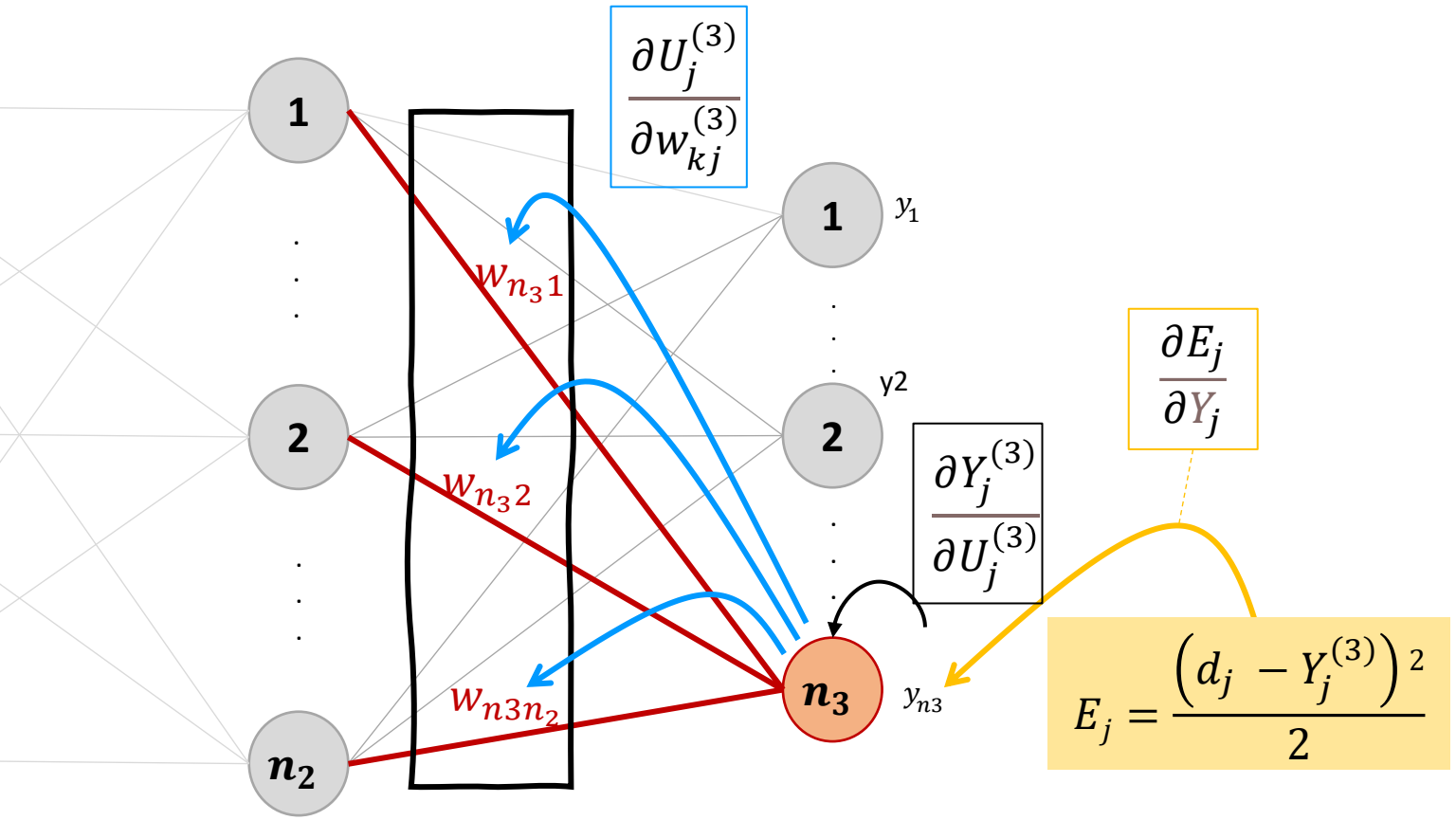
b = número sequencial da batelada

P = tamanho da batelada

s = iteração de início da batelada

$$\boldsymbol{w}(\boldsymbol{b}) = \boldsymbol{w}(\boldsymbol{b}-\mathbf{1}) - \boldsymbol{\alpha} \sum_{\boldsymbol{i}=\boldsymbol{s}}^{\boldsymbol{s}+\boldsymbol{P}-\mathbf{1}} \boldsymbol{\Delta w}(\boldsymbol{i})$$

MLP: Backpropagation para cam. de saída



Gradiente do erro pela regra da cadeia

$$\nabla E^{(3)} = \frac{\partial E_j}{\partial w_{kj}} = \frac{\partial E_j}{\partial Y_j^{(3)}} \cdot \frac{\partial Y_j^{(3)}}{\partial U_j^{(3)}} \cdot \frac{\partial U_j^{(3)}}{\partial W_{ji}^{(3)}} \quad (1)$$

$$= -\left(d_j - Y_j^{(3)}\right) . g'\left(U_j^{(3)}\right) . Y_I^{(2)}$$

Gradiente local em relação ao j-ésimo neurônio da saída

$$\delta_j^{(3)} = \left(d_j - Y_j^{(3)}\right) . g'\left(U_j^{(3)}\right)$$

Ajuste dos pesos na direção oposta ao gradiente

$$\Delta W_{ji}^{(3)} = -\eta . \frac{\partial E_j}{\partial W_{kj}} = \eta . \delta_j^{(3)} . Y_i^{(2)}$$

MLP: Algoritmo BP - ajuste pesos para as camadas (notação algorítmica)

Camada de saída (n=3)

$$\Delta \boldsymbol{W}_{\boldsymbol{ji}}^{(\boldsymbol{3})} \coloneqq \boldsymbol{W}_{\boldsymbol{ji}}^{(\boldsymbol{3})} + \boldsymbol{\eta}.\boldsymbol{\delta}_{\boldsymbol{j}}^{(\boldsymbol{3})}.\boldsymbol{Y}_{\boldsymbol{i}}^{(\boldsymbol{2})}$$

$$\delta_j^{(3)} = \left(d_j - Y_j^{(3)}\right).g'\left(U_j^{(3)}\right)$$

2ª. camada escondida (n=2)

$$\Delta \boldsymbol{W}_{\boldsymbol{ji}}^{(\boldsymbol{2})} \coloneqq \boldsymbol{W}_{\boldsymbol{ji}}^{(\boldsymbol{2})} + \boldsymbol{\eta}.\boldsymbol{\delta}_{\boldsymbol{j}}^{(\boldsymbol{2})}.\boldsymbol{Y}_{\boldsymbol{i}}^{(\boldsymbol{1})}$$

$$\delta_j^{(2)} = \left(\sum_{k=1}^{n_3} \delta_k^{(3)}.W_{kj}^{(3)}\right).g'\left(U_j^{(2)}\right)$$

1ª. camada escondida (n=1)

$$\Delta \boldsymbol{W}_{\boldsymbol{ji}}^{(\boldsymbol{1})} \coloneqq \boldsymbol{W}_{\boldsymbol{ji}}^{(\boldsymbol{1})} + \boldsymbol{\eta}.\boldsymbol{\delta}_{\boldsymbol{j}}^{(\boldsymbol{1})}.\boldsymbol{X}_{\boldsymbol{i}}^{(\boldsymbol{1})}$$

$$\delta_j^{(1)} = \left(\sum_{k=1}^{n_2} \delta_k^{(2)}.W_{kj}^{(2)}\right).g'\left(U_j^{(1)}\right)$$

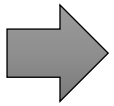
Vídeo explicativo

MLP: **algoritmo backpropagation com momento**

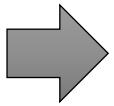
Problema do BP puro
sujeito a ficar preso nos mínimos locais

Solução
Utilizar o *backpropagation* com $\boldsymbol{momento\ \gamma}$ **para tornar a convergência mais eficiente**

Intuitivamente:

A solução atual está muito distante da solução final 

Quando a diferença entre o erro quadrático de duas iterações sucessivas é grande, o momento faz com que o passo em direção ao mínimo da função de erro seja grande.

A solução atual está próxima da solução final 

Quando a diferença é pequena, a contribuição do momento à convergência será pequena.

ver Momentum (at distill)

DICAS

REDUÇÃO DE OVERFITTING, CONVERGÊNCIA MAIS RÁPIDA

MLP: Algoritmo backpropagation dicas

PESOS	**Treinamento por batelada ou por padrão** Inicializar aleatoriamente os pesos no intervalo [-1,1]
BIAS LIMIAR	**Lembrar de incluir os limiares no conjunto de pesos** associados com entradas fixa em -1
ENTRADAS	**Normalizar as entradas:** [0.1, 0.9] sigmoide [-0.9, 0.9] tangente hiperbólica
TX APREND.	Utilizar valores pequenos **para taxa de aprendizado** $\eta \in [0.01, 0.1]$

MLP: Redução de Overfitting

Algumas técnicas para reduzir overfitting

Dropout: elimina neurônios de diferentes camadas de modo aleatório para evitar que o aprendizado se concentre em alguns poucos que se tornam extremamente especializados.

Regularização L1 e L2: limita a norma dos pesos - tende a aproximar os pesos entre si, i.e. não deixa os pesos ficarem tão discrepantes (SciKit Learn: regularização)

Utilizar método de validação cruzada para realizar experimentos (ver slides do conjunto 005 e este link do ScikitLearn para funcionamento específico na biblioteca)

MLP: Normalização das features

Facilita a convergência da Rede Neural.

Normalizar no pré-processamento: converter os valores das entradas e das saídas para um intervalo compatível ao da função de ativação utilizado na MLP (respectivamente, das camadas intermediárias e da saída).

Escala Min-Max
Para sigmoide $X_{min} = 0, X_{max} = 1$
Para tanH $X_{min} = -1, X_{max} = 1$

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Na fase de operação: quando for usar o valor de saída em uma aplicação real, não esquecer de desnormalizar

Outros ajustes de escalas: Robust, Power transformer, MaxAbs (ver slides 005)

TREINAMENTO: RESUMO

REDES NEURAIS

TIPOS DE APRENDIZADO

Supervisionado

- Correção prop. ao erro — Função ativação não-derivável — *Regra Delta*
- Correção prop. ao Gradiente do erro — Função ativação derivável — *F. ativação derivável (ex. Backpropagation)*

Não-supervisionado

- Hebbiano — um neurônio contribui para o disparo de outro, a conexão entre eles se fortalece
- Por competição — o ajuste leva o peso do vencedor *k* na direção da entrada

Por reforço

a saída não é conhecida a priori, mas pode-se avaliar se a ação é positiva ou negativa (em relação a um objetivo) resultando respectivamente em recompensa e punição

Arquiteturas de RNs

Arquiteturas das Redes Neurais

Número de camadas

SINGLE-LAYER	só a camada de entrada e de saída
MULTI-LAYER	número de camadas escondidas (não contar a de entrada e a de saída)

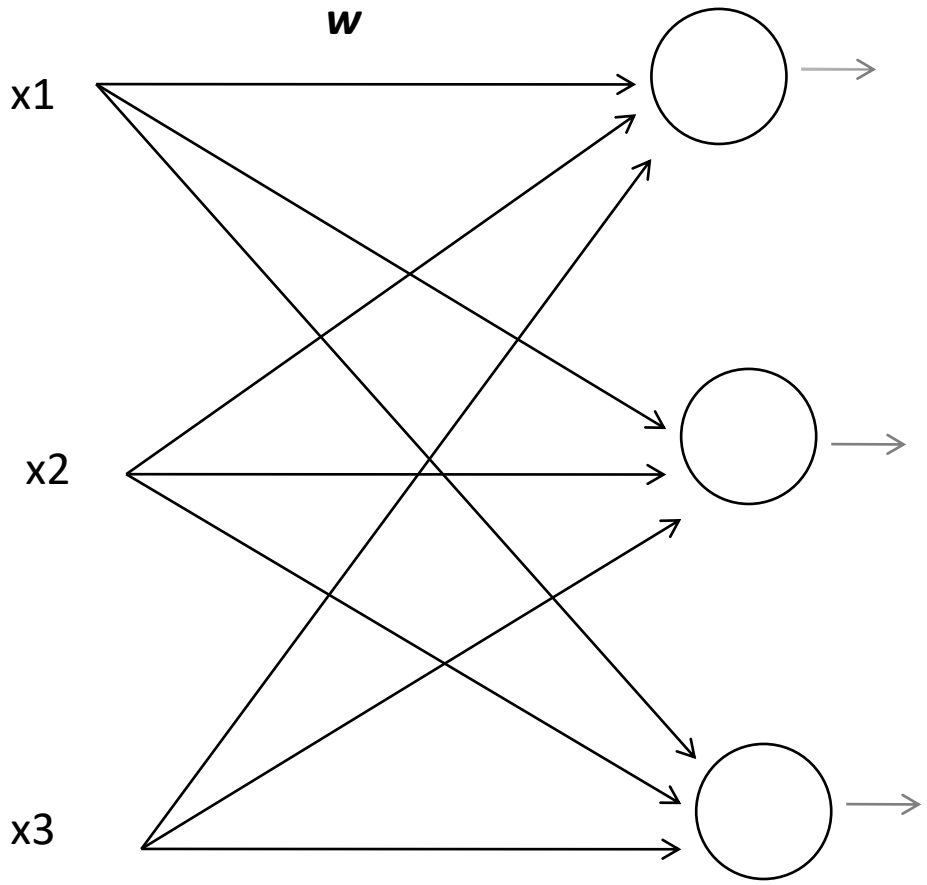
Tipo de conexão entre camadas

FEEDFORWARD	um neurônio de uma camada n não se conecta a neurônios de camadas anteriores
COM FEEDBACK	(cíclicas ou recorrentes): um ou mais neurônios de uma camada n estão conectados a neurônios de camadas anteriores

Conectividade

TOTALMENTE	cada neurônio da camada n está ligado a todos os neurônios da camada n-1
PARCIALMENTE	a restrição acima não é verificada

Arquitetura Perceptron

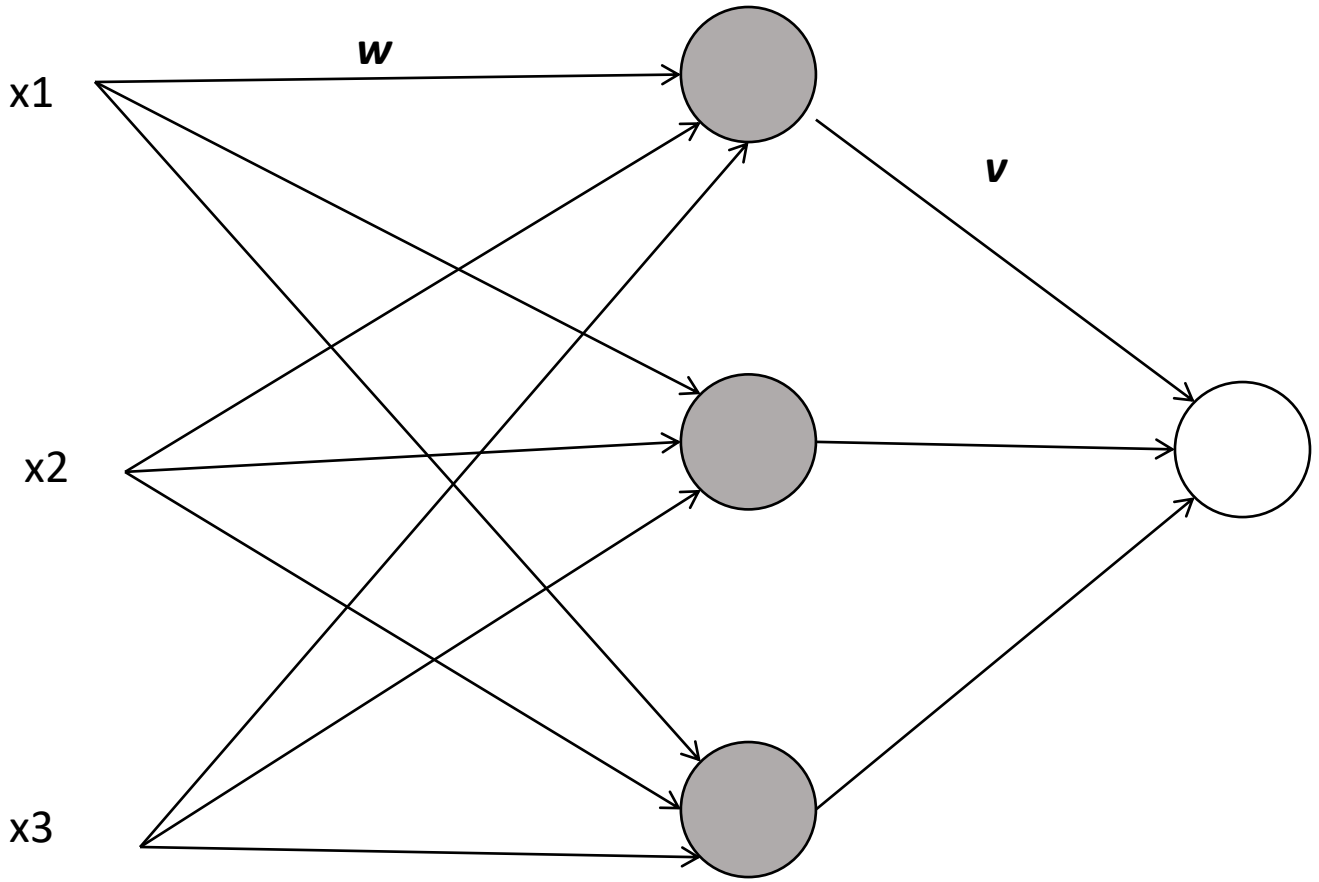


Número de camadas: **única**

Tipo de conectividade: **feedforward**

Conectividade: **totalmente conectada**

Arquitetura Multi-layer Perceptron

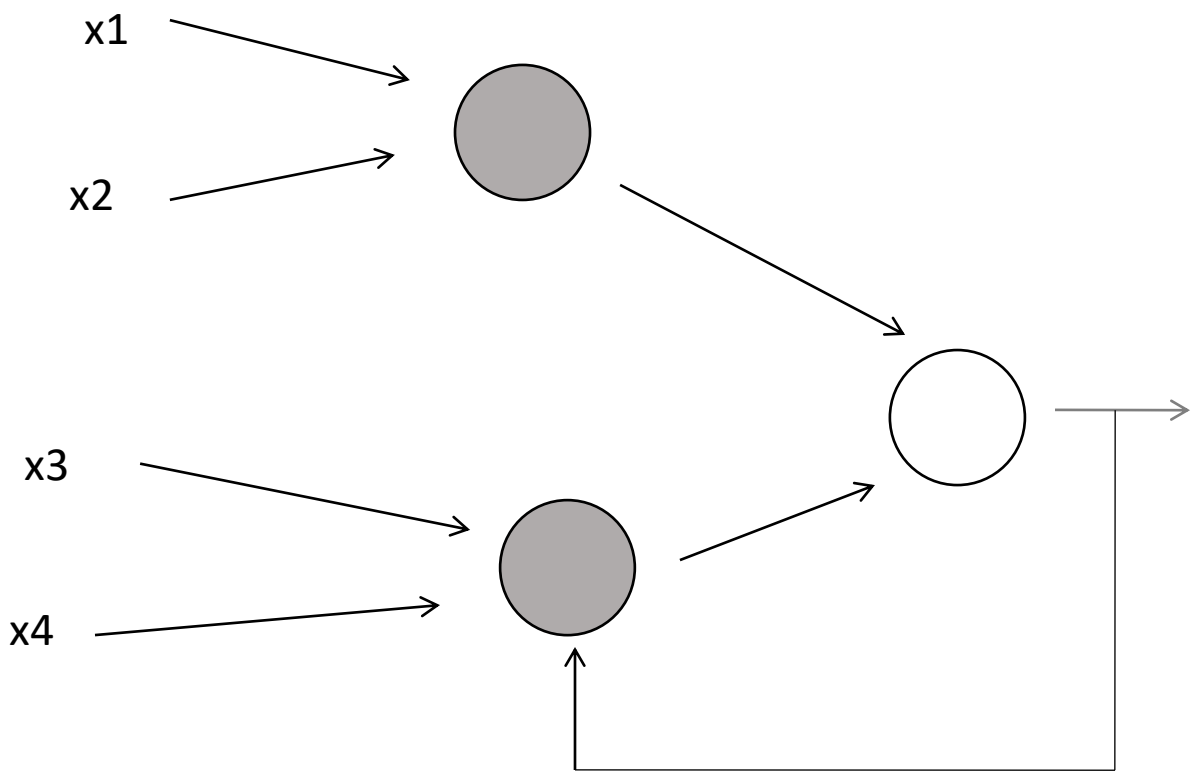


Número de camadas: **MULTI**
com 1 camada escondida

Tipo de conectividade: **FEEDFORWARD**

Conectividade: **TOTAL**

Arquitetura recorrente

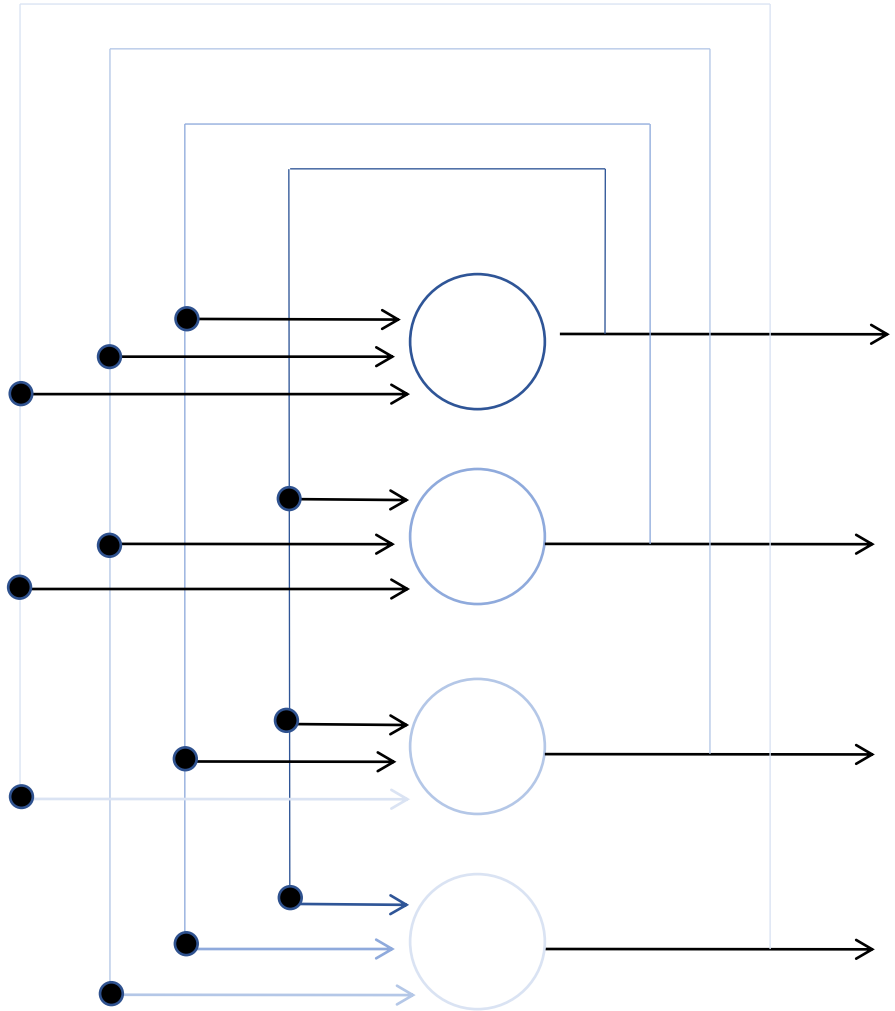


Número de camadas: **MÚLTIPLAS (1 escondida)**

Tipo de conectividade: **RECORRENTE**

Conectividade: **PARCIAL**

Arquitetura de Rede de Hopfield



Número de camadas: **ÚNICA**

Tipo de conectividade: **RECORRENTE**

Conectividade: **PARCIAL**